In-Class Solutions

Instructor key. Algorithms & Data Structures, Professor Roger Hunt

This key gives, for each of the 60 board problems, a solution that a student can write at the board, an explanation of how it works, its time and auxiliary space, a memory estimate at the input size printed on the student's sheet, and other approaches with their cost.

Every board solution in this key has been run on Python 3.13 against LeetCode's examples, against edge cases at LeetCode's limits, and against random inputs checked by a slower brute-force solution, and every one returned correct results. The test scripts are in the `tests` folder beside this document, so you can run them again. Every memory figure was recomputed from the course's table of sizes, and most were also compared with measurements from `tracemalloc`. Those measurements run about 10 percent higher on inputs full of integers, because `tracemalloc` counts 32 bytes for each integer rather than 28, and they miss recursive calls entirely, because `tracemalloc` does not count call frames.

How to use this key

Credit. A correct board problem earns 50 points of extra credit, and every 100 points make up one missed class. An answer earns that credit only if the student can explain and defend it. Ask why the solution works, why its complexity is what the student wrote, and what would change for a different input.

Auxiliary space. Auxiliary space is the extra memory a solution uses beyond its input, and it includes anything the solution builds and returns. Where the output dominates, as in Subsets or N-Queens, the key says so.

Memory estimates. The memory figures are estimates. Accept any answer within about 30 percent of the key's figure when the student writes down sensible assumptions, such as a different n, a differently shaped input or a different version of Python. Grade the method: the student should count list pointers and object sizes, use the right n, and count the input separately from the auxiliary data. KB means 1,024 bytes and MB means 1,048,576 bytes.

Other approaches. Each problem lists other approaches with their cost. Accept any of them if the student analyzes it correctly.

Asking for help. A student who asks for help must first show two failed attempts at the bottom of the answer sheet.

Recursion. Python stops a recursion at a depth of 1,000 unless the limit is raised. Where a recursive solution can exceed that depth at LeetCode's limits, the key says so and names a version without recursion.

Memory sizes in CPython 3.13 on a 64-bit machine

Object	Approximate size
int	28 bytes below 2^{30}, about one billion, and 4 bytes more for each further 30 bits. The integers from −5 to 256 are shared and cost nothing extra.
float	24 bytes.
True, False and None	Shared, so only the 8-byte pointer to them counts.
str	41 bytes plus 1 byte per character for plain ASCII text. A one-character string is

Object	Approximate size
	shared when the character is one of the first 256.
list	56 bytes plus 8 bytes per slot. `[0] * n` and `list(...)` give exactly n slots; a list built with `append` or a comprehension keeps up to about 13 percent spare slots, about 6 percent on average.
tuple	40 bytes plus 8 bytes per item: 56 for a pair and 64 for a triple.
dict	64 bytes when empty, and 184 bytes with one to five string keys. With integer keys, about 37 bytes per entry at 1,000 entries, 30 at 10,000 and 52 at 100,000; with string keys, about 26 at 1,000 and 38 at 100,000.
set	At least 216 bytes. About 33 bytes per element at 1,000 elements and 42 at 100,000.
deque	About 760 bytes plus 8.3 bytes per element.
node object	About 90 bytes for a ListNode and 100 for a TreeNode, plus 28 bytes for a value outside −5 to 256.
recursive call	About 150 to 250 bytes for each call that has not yet returned, more for a function with many local variables.

A dict or a set grows by building a bigger table and copying itself into it, so at large sizes it briefly holds about 1.5 times its final table. Lists and deques grow without such a jump.

#1480 Running Sum of 1d Array

Easy. https://leetcode.com/problems/running-sum-of-1d-array/

You are given a list of integers, `nums`. Return a list in which entry `i` is the sum of `nums[0]` through `nums[i]`. This list is called the running sum.

For example, `nums = [1, 2, 3, 4]` gives `[1, 3, 6, 10]`.

Start with `def runningSum(nums):`

Estimate the memory for n = 100,000 distinct integers, all above 256, so that none of them is one of Python's shared small integers.

Solution

```
def runningSum(nums):
    for i in range(1, len(nums)):
        nums[i] = nums[i] + nums[i - 1]
    return nums
```

How the solution works

1. Entry 0 is already correct, because its running sum is the entry itself.
2. For each later position i, the running sum is the previous running sum plus `nums[i]`. The previous entry already holds that total, so the solution adds it in.
3. The loop works from left to right, so `nums[i - 1]` is finished before the solution uses it.
4. The function returns the same list. It changed the list in place, so it never builds a second list.

Time and space

Time: O(n). The loop runs n − 1 times and does a constant amount of work each time.

Auxiliary space: O(1). The solution keeps only the index `i` and the loop's iterator, and it writes each sum into the input list itself.

Memory estimate

The input list holds 100,000 pointers and 100,000 integers. The auxiliary data is the index and the iterator, about 150 bytes, but the sums add a cost that is easy to miss. Because the values are distinct and all above 256, the sums grow quickly: even the smallest possible values, 257 through 100,256, add up to more than five billion. Once a running sum passes 2^{30}, which is about 1.07 billion, Python stores it in 32 bytes rather than 28. Most entries of the list are therefore replaced by an integer 4 bytes larger, the list's integers grow by roughly 0.3 to 0.4 MB depending on the values, and that growth remains after the function returns.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	index and iterator, about 150 bytes, plus 4 bytes for each running sum of 2^{30} or more	up to about 0.4 MB
Total	input plus auxiliary	about 3.8 MB

Accept about 150 bytes of auxiliary data from a student who assumes that every sum stays below 2^{30}, and then ask how large the sums really get. The question shows that a Python integer is not a fixed-size object. LeetCode's own limits, n up to 1,000 and values no larger than 10^6 in size, keep every sum below 2^{30}, so the extra cost appears only at the larger n used here.

Other approaches

Approach	Time	Auxiliary space
Build a new list with a loop and `append`	O(n)	O(n)
`itertools.accumulate`, which is not allowed at the board	O(n)	O(n)

#217 Contains Duplicate

Easy. https://leetcode.com/problems/contains-duplicate/

You are given a list of integers, `nums`. Return `True` if any value appears at least twice in the list, and `False` if every value is distinct.

For example, `nums = [1, 2, 3, 1]` gives `True`, and `nums = [1, 2, 3, 4]` gives `False`.

Start with `def containsDuplicate(nums):`

Estimate the memory for n = 100,000 distinct integers, all above 256.

Solution

```
def containsDuplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

How the solution works

1. The brute-force approach compares every pair of values. That takes about $n(n-1)/2$ comparisons, which is $O(n^2)$ time, with O(1) extra space.
2. A better approach remembers the values it has already seen. A set answers the question “have I seen x before?” in O(1) time on average.
3. The solution walks the list once. If x is already in the set, it has found a duplicate and returns `True`; otherwise it adds x to the set.
4. If the loop finishes, every value was new, so the function returns `False`.
5. The solution spends O(n) memory to cut the time from $O(n^2)$ to O(n).

Sets are not taught until Class 4. A student in Class 1 may reach for the nested loops or for sorting, both listed under “Other approaches”; accept either one with a correct analysis.

Time and space

Time: O(n). The loop takes n steps, and each step is a set lookup and a set insertion that take O(1) time on average.

Auxiliary space: O(n). When every value is distinct, the set ends up holding all n values.

Memory estimate

The set stores pointers to the integers that already exist in the input, so it creates no new integers. A set of 100,000 elements costs about 42 bytes per element: a 216-byte header plus a table of 2^{18} = 262,144 slots of 16 bytes each, which `sys.getsizeof` reports as 4,194,520 bytes.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	a set of 100,000 elements, at about 42 bytes per element	4.0 MB
Total	input plus auxiliary	7.4 MB

A set grows by building a bigger table and copying itself into it. During the last copy it briefly holds both tables, so its peak is about 1.5 times the final table, about 6 MB.

Other approaches

Approach	Time	Auxiliary space
Brute force: compare every pair with two nested loops	$O(n^2)$	O(1)
Sort, then compare neighbors	O(n log n)	O(n) in Python, whose sort uses a buffer of up to n/2 pointers
A set (the solution above)	O(n)	O(n)

Class 2: Arrays and dynamic arrays. Problem A, the easier problem.

#283 Move Zeroes

Easy. https://leetcode.com/problems/move-zeroes/

You are given a list of integers, `nums`. Move every 0 to the end of the list while keeping the non-zero values in their original order. You must change the list in place, without making a copy.

For example, `nums = [0, 1, 0, 3, 12]` becomes `[1, 3, 12, 0, 0]`.

The function returns nothing; it changes `nums` itself. Start with `def moveZeroes(nums):`

Estimate the memory for n = 100,000 integers, half of them 0. That is ten times LeetCode's limit, which makes the numbers easier to see.

Solution

```
def moveZeroes(nums):
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write], nums[read] = nums[read], nums[write]
            write += 1
```

How the solution works

1. The solution uses two indexes. `read` visits every position, and `write` marks where the next non-zero value belongs.
2. Every position before `write` is finished: it holds non-zero values in their original order.
3. When `nums[read]` is not zero, the solution swaps it into position `write` and moves `write` forward by one.
4. The solution never moves a zero forward on purpose. The swaps push the zeros toward the back.
5. The function returns nothing, because it changes the list itself.

Time and space

Time: O(n). `read` visits each position once and makes at most one swap at each.

Auxiliary space: O(1). The solution keeps two indexes.

Memory estimate

Every 0 in the list is the same shared object, so only the 50,000 non-zero values cost 28 bytes each. The swaps only move pointers around.

Part	How it is counted	Estimate
Input	56 + 8n + 28 × n/2	2.1 MB
Auxiliary	two indexes and the loop's iterator	about 150 bytes
Total	input plus auxiliary	2.1 MB

Other approaches

Approach	Time	Auxiliary space
Copy the non-zero values to a new list, add the zeros, and copy the result back	O(n)	O(n)

Approach	Time	Auxiliary space
Remove each zero and append it at the end	$O(n^2)$	$O(1)$
Write the non-zero values forward, then fill the rest of the list with zeros	$O(n)$	$O(1)$

#88 Merge Sorted Array

Easy. https://leetcode.com/problems/merge-sorted-array/

You are given two sorted lists. `nums1` has length m + n: its first m entries are values in sorted order, and its last n entries are zeros that serve as empty room. `nums2` holds n values in sorted order. Merge `nums2` into `nums1` so that `nums1` holds all m + n values in sorted order. Change `nums1` in place.

For example, `nums1 = [1, 2, 3, 0, 0, 0]`, `m = 3`, `nums2 = [2, 5, 6]` and `n = 3` turn `nums1` into `[1, 2, 2, 3, 5, 6]`.

The function returns nothing; it changes `nums1` itself. Start with `def merge(nums1, m, nums2, n):`

Estimate the memory for m = n = 50,000, so that `nums1` has 100,000 slots and `nums2` has 50,000. The values are distinct integers above 256. LeetCode's limit is m + n ≤ 200; the larger sizes make the numbers easier to see.

Solution

```
def merge(nums1, m, nums2, n):
    i = m - 1
    j = n - 1
    k = m + n - 1
    while j >= 0:
        if i >= 0 and nums1[i] > nums2[j]:
            nums1[k] = nums1[i]
            i -= 1
        else:
            nums1[k] = nums2[j]
            j -= 1
        k -= 1
```

How the solution works

1. Merging from the front would overwrite values of `nums1` that the solution still needs.
2. The empty room is at the back, so the solution fills `nums1` from the back, writing the largest remaining value into position `k`.
3. `i` points at the last unused value of `nums1`, and `j` at the last unused value of `nums2`.
4. At each step the solution compares `nums1[i]` with `nums2[j]`, writes the larger one at position `k`, and moves that pointer and `k` one place to the left.
5. The loop stops when `nums2` is used up. Any values left in `nums1` are already in place.

Time and space

Time: O(m + n). Each step writes one position, and there are at most m + n positions to write.

Auxiliary space: O(1). The solution keeps three indexes.

Memory estimate

The 50,000 zeros at the end of `nums1` are the shared integer 0, so they cost only their pointers.

Part	How it is counted	Estimate
Input	`nums1`: 56 + 8 × 100,000 + 28 × 50,000; `nums2`: 56 + 8 × 50,000 + 28 × 50,000	3.8 MB
Auxiliary	three indexes	about 150 bytes

Part	How it is counted	Estimate
Total	input plus auxiliary	3.8 MB

Other approaches

Approach	Time	Auxiliary space
Copy `nums2` into the empty room, then sort `nums1`	O((m + n) log(m + n))	O(m + n) in Python, because of the sort's buffer
Merge from the front into a new list, then copy it back	O(m + n)	O(m + n)
Merge from the back (the solution above)	O(m + n)	O(1)

#14 Longest Common Prefix

Easy. https://leetcode.com/problems/longest-common-prefix/

You are given a list of strings, `strs`. Return the longest string that is a prefix of every string in the list. If no such string exists, return the empty string `""`.

For example, `strs = ["flower", "flow", "flight"]` gives `"fl"`, and `strs = ["dog", "racecar", "car"]` gives `""`.

Start with `def longestCommonPrefix(strs):`

Estimate the memory for m = 200 strings of 200 characters each, all identical. This is the worst case that LeetCode's limits allow.

Solution

```
def longestCommonPrefix(strs):
    first = strs[0]
    for i in range(len(first)):
        for s in strs:
            if i == len(s) or s[i] != first[i]:
                return first[:i]
    return first
```

How the solution works

1. The answer can be no longer than the first string, so the solution uses the first string as a template.
2. It checks one column at a time: at position i, every string must have the same character as `first`.
3. The first time a string is too short or has a different character at position i, the prefix is `first[:i]`.
4. If the solution gets through all of `first` without a mismatch, the whole first string is the prefix.
5. The solution slices only once, at the end, so it pays for copying only once.

Time and space

Time: O(S), where S is the total number of characters in all the strings, at most m × L for strings of length L. In the worst case the solution reads every character of every string once.

Auxiliary space: O(L). The only thing the solution builds is the returned prefix, which is at most as long as the shortest string.

Memory estimate

When all the strings are identical, the loop never finds a mismatch, so the function returns `first` itself and copies nothing. Only a few indexes remain. If a mismatch had occurred near the end, the returned slice would add up to about 240 bytes.

Part	How it is counted	Estimate
Input	200 strings × (41 + 200) bytes, plus a list of 200 pointers, 56 + 8 × 200	49 KB
Auxiliary	a few indexes; no slice, because the function returns the input string itself	about 150 bytes
Total	input plus auxiliary	49 KB

Other approaches

Approach	Time	Auxiliary space
Shorten a running prefix against each string in turn	O(S)	O(L)
Sort the strings, then compare only the first and the last	O(L × m log m), because each comparison can read L characters	O(m)

Class 3: Strings and prefix sums. Problem B, the harder problem.

#303 Range Sum Query - Immutable

Easy. https://leetcode.com/problems/range-sum-query-immutable/

Design a class `NumArray`. It is built once from a list of integers, `nums`. After that, `sumRange(left, right)` is called many times, and each call must return the sum of `nums[left]` through `nums[right]`, including both ends.

For example, after `NumArray([-2, 0, 3, -5, 2, -1])`, the call `sumRange(0, 2)` returns 1, `sumRange(2, 5)` returns −1, and `sumRange(0, 5)` returns −3.

Start with `class NumArray:` and write the methods `__init__(self, nums)` and `sumRange(self, left, right)`.

Estimate the memory for n = 10,000 values, which is LeetCode's limit, with values large enough that every prefix sum is a new integer above 256.

Solution

```
class NumArray:
    def __init__(self, nums):
        self.prefix = [0]
        for x in nums:
            self.prefix.append(self.prefix[-1] + x)

    def sumRange(self, left, right):
        return self.prefix[right + 1] - self.prefix[left]
```

How the solution works

1. Adding up the range on every call costs O(n) per call, which is slow when there are many calls.
2. The solution does the work once instead. `prefix[i]` holds the sum of the first i numbers, and `prefix[0]` is 0.
3. It builds the prefix list in one pass: each entry is the previous entry plus the next number.
4. The sum of `nums[left]` through `nums[right]` is everything up to `right` minus everything before `left`, which is `prefix[right + 1] - prefix[left]`.
5. The extra 0 at the front means that `left = 0` needs no special case.

Time and space

Time: O(n) to build, and O(1) for each call to `sumRange`. Building takes one pass of n steps, and each call does two lookups and one subtraction.

Auxiliary space: O(n). The prefix list holds n + 1 numbers.

Memory estimate

The prefix list is built with `append`, so it carries some spare slots. At this size `sys.getsizeof` reports 85,176 bytes for it, about 6 percent more than its 10,001 pointers need. Each of the 10,000 sums is a new integer.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	352 KB
Auxiliary	prefix list of 10,001 pointers plus spare slots (83 KB), plus 10,000 new integers × 28 bytes	357 KB

Part	How it is counted	Estimate
Total	input plus auxiliary	709 KB

Other approaches

Approach	Time	Auxiliary space
Add up the range on every call	O(1) to build, O(n) per call	O(1)
Store the answer for every possible range	$O(n^2)$ to build, O(1) per call	$O(n^2)$
A prefix-sum list (the solution above)	O(n) to build, O(1) per call	O(n)

Class 4: Hash tables. Problem A, the easier problem.

#242 Valid Anagram

Easy. https://leetcode.com/problems/valid-anagram/

You are given two strings, `s` and `t`, made of lowercase letters. Return `True` if `t` is an anagram of `s`, which means that it uses exactly the same letters the same number of times, and `False` otherwise.

For example, `s = "anagram"` and `t = "nagaram"` give `True`, and `s = "rat"` and `t = "car"` give `False`.

Start with `def isAnagram(s, t):`

Estimate the memory for n = 50,000 characters in each string, which is LeetCode's limit.

Solution

```
def isAnagram(s, t):
    if len(s) != len(t):
        return False
    counts = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1
    for ch in t:
        if counts.get(ch, 0) == 0:
            return False
        counts[ch] -= 1
    return True
```

How the solution works

1. If the lengths differ, the answer is `False` immediately.
2. The solution counts each letter of `s` in a dict that maps a letter to the number of times it appears.
3. It then walks through `t` and uses up one count for each letter. If a letter has no count left, `t` has too many of that letter, and the answer is `False`.
4. The lengths are equal and no count went below zero, so every count ends at zero, and the answer is `True`.
5. The dict holds at most 26 keys, so its size does not grow with n.

Time and space

Time: O(n). The solution makes two passes over strings of length n, and each step is a dict operation that takes O(1) time on average.

Auxiliary space: O(1). The dict holds at most 26 letters, however long the strings are.

Memory estimate

The one-letter keys are shared strings, so they cost nothing extra. A dict with 26 string keys is 832 bytes. The counts grow past 256, so each of the 26 final counts is a new integer.

Part	How it is counted	Estimate
Input	2 strings × (41 + n)	98 KB
Auxiliary	dict with 26 keys (832 bytes) plus 26 counts × 28 bytes	1.5 KB
Total	input plus auxiliary	99 KB

Other approaches

Approach	Time	Auxiliary space
Sort both strings and compare them	O(n log n)	O(n)
A list of 26 counters, one for each letter	O(n)	O(1)

Class 4: Hash tables. Problem B, the harder problem.

#1 Two Sum

Easy. https://leetcode.com/problems/two-sum/

You are given a list of integers, `nums`, and an integer, `target`. Return the indexes of the two numbers that add up to `target`. Exactly one such pair exists, and you may not use the same position twice.

For example, `nums = [2, 7, 11, 15]` and `target = 9` give `[0, 1]`.

Start with `def twoSum(nums, target):`

Estimate the memory for n = 10,000 distinct integers above 256, which is LeetCode's limit, with the matching pair at the very end, so that the dict grows to n − 1 entries.

Solution

```
def twoSum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
```

How the solution works

1. The brute-force approach tries every pair, which takes O(n²) time and O(1) space.
2. For each value x, the partner it needs is `target - x`. Instead of searching for that partner, the solution remembers every earlier value in a dict that maps a value to its index.
3. At position i the solution computes `need = target - x`. If `need` is already in the dict, it returns the stored index and i.
4. Otherwise it stores x with its index and moves on. It stores x only after checking, so it never pairs x with itself.
5. One pass and one dict give O(n) time and O(n) space.

Time and space

Time: O(n). The solution makes one pass, and each dict operation takes O(1) time on average.

Auxiliary space: O(n). In the worst case the pair is at the very end, and the dict holds n − 1 entries.

Memory estimate

The keys point to integers that already exist in the input. Each stored index above 256 is a new integer, and there are 9,742 of them. A dict of 9,999 integer keys costs about 30 bytes per entry, which `sys.getsizeof` reports as 294,992 bytes.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	352 KB
Auxiliary	dict of 9,999 entries at about 30 bytes each (288 KB), plus 9,742 new index integers × 28 bytes (266 KB)	554 KB
Total	input plus auxiliary	906 KB

Other approaches

Approach	Time	Auxiliary space
Brute force: try every pair	$O(n^2)$	O(1)
Sort (value, index) pairs, then move two pointers inward	O(n log n)	O(n)

Class 5: Two pointers and sliding windows. Problem A, the easier problem.

#125 Valid Palindrome

Easy. https://leetcode.com/problems/valid-palindrome/

A phrase is a palindrome if, after you turn every uppercase letter into lowercase and remove every character that is not a letter or a digit, it reads the same forward and backward. Given a string `s`, return `True` if it is a palindrome and `False` otherwise.

For example, `s = "A man, a plan, a canal: Panama"` gives `True`, and `s = "race a car"` gives `False`.

Start with `def isPalindrome(s):`

Estimate the memory for n = 200,000 characters, which is LeetCode's limit, all letters, forming a palindrome. This is the worst case, because the solution checks all the way to the middle.

Solution

```
def isPalindrome(s):
    left = 0
    right = len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

How the solution works

1. The solution puts one pointer at each end of the string and moves them toward each other.
2. It skips any character that is not a letter or a digit, advancing `left` or pulling back `right` until both point at characters that count.
3. It compares the two characters, ignoring case. A mismatch means the answer is `False`.
4. On a match it moves both pointers inward. When the pointers meet or cross, every pair has matched, and the answer is `True`.
5. Cleaning the string first and comparing it with its reverse also works, but it copies the string and costs O(n) extra space.

Time and space

Time: O(n). Each pointer moves only inward, so together they make at most n moves.

Auxiliary space: O(1). The solution keeps two indexes. Each `s[i]` is a shared one-character string. Each call to `lower()` creates a new one-character string, but Python frees it at once, so only a few exist at any moment.

Memory estimate

Part	How it is counted	Estimate
Input	41 + n	195 KB
Auxiliary	two indexes and a few temporary strings	about 150 bytes

Part	How it is counted	Estimate
Total	input plus auxiliary	195 KB

Other approaches

Approach	Time	Auxiliary space
Clean the string, then compare it with its reverse	O(n)	O(n)

Class 5: Two pointers and sliding windows. Problem B, the harder problem.

#643 Maximum Average Subarray I

Easy. https://leetcode.com/problems/maximum-average-subarray-i/

You are given a list of integers, `nums`, and an integer, `k`. Find the run of k neighboring values that has the largest average, and return that average.

For example, `nums = [1, 12, -5, -6, 50, 3]` and `k = 4` give 12.75, the average of `[12, -5, -6, 50]`.

Start with `def findMaxAverage(nums, k):`

Estimate the memory for n = 100,000 integers above 256, which is LeetCode's limit. The running sum is a new integer at each step, but only one is alive at a time.

Solution

```
def findMaxAverage(nums, k):
    window = 0
    for i in range(k):
        window += nums[i]
    best = window
    for i in range(k, len(nums)):
        window += nums[i] - nums[i - k]
        if window > best:
            best = window
    return best / k
```

How the solution works

1. Because k is fixed, the window with the largest average is also the window with the largest sum. The solution finds the largest sum of k neighboring values and divides by k at the end.
2. The brute-force approach adds up every window from scratch, which takes about n × k steps.
3. The solution adds up the first k numbers once.
4. To slide the window one step to the right, it adds the entering number, `nums[i]`, and subtracts the leaving number, `nums[i - k]`.
5. It keeps the best sum it has seen and returns `best / k`.

Time and space

Time: O(n). The first window takes k steps, and each of the n − k slides does a constant amount of work.

Auxiliary space: O(1). The solution keeps `window`, `best` and `i`.

Memory estimate

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	a few integers and one float	about 150 bytes
Total	input plus auxiliary	3.4 MB

Other approaches

Approach	Time	Auxiliary space
Brute force: add up each window from scratch	O(nk)	O(1)
Prefix sums: each window is `prefix[i + k] - prefix[i]`	O(n)	O(n)

#20 Valid Parentheses

Easy. https://leetcode.com/problems/valid-parentheses/

You are given a string, `s`, that contains only the characters `(`, `)`, `[`, `]`, `{` and `}`. Return `True` if the string is valid and `False` otherwise. A string is valid when every opening bracket is closed by a bracket of the same type, the brackets close in the correct order, and every closing bracket has a matching opening bracket of the same type.

For example, `s = "()[]{}"` gives `True`, while `s = "(]"` and `s = "([)]"` give `False`.

Start with `def isValid(s):`

Estimate the memory for n = 10,000 characters, which is LeetCode's limit, all of them `(`. This is the worst case for the size of the stack.

Solution

```
def isValid(s):
    stack = []
    pairs = {')': '(', ']': '[', '}': '{'}
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

How the solution works

1. The most recent opening bracket must be closed first. That is last in, first out, which is exactly what a stack does.
2. The solution pushes every opening bracket onto the stack.
3. On a closing bracket it pops the stack. If the stack is empty, or the popped bracket is the wrong type, the answer is `False`.
4. The dict `pairs` maps each closing bracket to the opening bracket it needs.
5. At the end the stack must be empty; otherwise some opening bracket was never closed.

Time and space

Time: O(n). Each character is pushed or popped once.

Auxiliary space: O(n). When every character is an opening bracket, the stack holds all n of them.

Memory estimate

The one-character strings are shared, so the stack holds only pointers. A list built by 10,000 appends has some spare slots; `sys.getsizeof` reports 85,176 bytes for it. A dict with three string keys is 184 bytes.

Part	How it is counted	Estimate
Input	41 + n	9.8 KB
Auxiliary	stack of 10,000 pointers plus spare slots (83 KB), plus the three-entry dict (184 bytes)	83 KB
Total	input plus auxiliary	93 KB

Other approaches

Approach	Time	Auxiliary space
Repeatedly delete `"()"`, `"[]"` and `"{}"` from the string until nothing changes	$O(n^2)$	$O(n)$

Class 6: Stacks. Problem B, the harder problem.

#844 Backspace String Compare

Easy. https://leetcode.com/problems/backspace-string-compare/

Two strings, `s` and `t`, contain lowercase letters and the character `#`, which stands for a backspace: it deletes the character before it, and it does nothing if there is none. Return `True` if the two strings are equal once every backspace has been applied, and `False` otherwise.

For example, `s = "ab#c"` and `t = "ad#c"` give `True`, because both become `"ac"`, while `s = "a#c"` and `t = "b"` give `False`.

Start with `def backspaceCompare(s, t):`

Estimate the memory for two strings of 200 characters each, which is LeetCode's limit, with no # characters. This is the worst case for the size of the stacks.

Solution

```
def backspaceCompare(s, t):
    def build(text):
        stack = []
        for ch in text:
            if ch == '#':
                if stack:
                    stack.pop()
            else:
                stack.append(ch)
        return stack
    return build(s) == build(t)
```

How the solution works

1. Typing characters and pressing backspace behaves like a stack: each letter is pushed, and each `#` pops the most recent letter.
2. The helper `build(text)` returns the final stack of letters.
3. A `#` on an empty stack does nothing.
4. The solution builds both stacks and compares them with `==`.
5. LeetCode's follow-up asks for O(1) extra space. Two pointers that move backward from the end of each string can meet it.

Time and space

Time: O(n + m). Each character is processed once, and the final comparison takes at most min(n, m) steps.

Auxiliary space: O(n + m). Each stack can hold up to the full length of its string.

Memory estimate

The one-character strings are shared, so the stacks hold only pointers. A list built by 200 appends is 1,656 bytes.

Part	How it is counted	Estimate
Input	2 strings × (41 + 200)	482 bytes
Auxiliary	two lists of 200 pointers, 1,656 bytes each	3.2 KB
Total	input plus auxiliary	3.7 KB

Other approaches

Approach	Time	Auxiliary space
Two pointers moving backward from the end of each string, skipping deleted letters	O(n + m)	O(1)

#933 Number of Recent Calls

Easy. https://leetcode.com/problems/number-of-recent-calls/

Design a class `RecentCounter`. Its method `ping(t)` records a request at time t, measured in milliseconds, and returns how many requests have arrived in the last 3,000 milliseconds, from t − 3000 to t, including this one. Each call uses a larger t than the call before it.

For example, `ping(1)` returns 1, `ping(100)` returns 2, `ping(3001)` returns 3, and `ping(3002)` returns 3.

Start with `class RecentCounter:` and write the methods `__init__(self)` and `ping(self, t)`.

Estimate the memory for 10,000 pings, one every millisecond. This is the worst case for the size of the window, which then holds 3,001 entries.

Solution

```
from collections import deque

class RecentCounter:
    def __init__(self):
        self.window = deque()

    def ping(self, t):
        self.window.append(t)
        while self.window[0] < t - 3000:
            self.window.popleft()
        return len(self.window)
```

How the solution works

1. Only requests from the last 3,000 milliseconds matter. Older ones can be forgotten for good, because time only moves forward.
2. The solution keeps the recent request times in a queue, with the oldest at the front and the newest at the back.
3. `ping(t)` adds t at the back.
4. It then removes from the front every time that is older than t − 3000.
5. The length of the queue is the answer. A list would cost O(n) for each removal from the front with `pop(0)`, so the solution uses a `deque`, which needs the import on the first line.

Time and space

Time: O(1) amortized per ping. Each time is added once and removed once, so the total work is proportional to the number of pings.

Auxiliary space: O(W), where W is the number of pings that fit in the window. Here W is at most 3,001.

Memory estimate

The caller creates the times, which are integers, and the deque holds pointers to them. A deque of 3,001 entries is about 25 KB: `sys.getsizeof` reports 25,048 bytes.

Part	How it is counted	Estimate
Input	10,000 times created by the caller, × 28 bytes	273 KB
Auxiliary	deque of 3,001 entries	25 KB

Part	How it is counted	Estimate
Total	input plus auxiliary	298 KB

Other approaches

Approach	Time	Auxiliary space
Keep every time in a list and count the times in range on each ping	O(n) per ping	O(n)
Keep every time in a list and find the start of the window by binary search	O(log n) per ping	O(n)
Use a list as the queue and remove old times with `pop(0)`	O(W) per removal	O(W)

#232 Implement Queue using Stacks

Easy. https://leetcode.com/problems/implement-queue-using-stacks/

Implement a first-in, first-out queue using only two stacks. The stacks are Python lists that you may use only with `append`, `pop()` from the end, `[-1]` to look at the end, and `len`. Provide the methods `push(x)`, `pop()`, `peek()` and `empty()`. Every call to `pop` and `peek` is made on a queue that is not empty.

For example, after `push(1)` and `push(2)`, the call `peek()` returns 1, `pop()` returns 1, and `empty()` returns `False`.

Start with `class MyQueue:` and write the methods `__init__(self)`, `push(self, x)`, `pop(self)`, `peek(self)` and `empty(self)`.

Estimate the memory for 100,000 pushes followed by pops. This is far more than LeetCode's limit of 100 calls, and it makes the growth visible. Assume that each item is a separate integer object that the caller created.

Solution

```
class MyQueue:
    def __init__(self):
        self.inbox = []
        self.outbox = []

    def push(self, x):
        self.inbox.append(x)

    def pop(self):
        self.peek()
        return self.outbox.pop()

    def peek(self):
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())
        return self.outbox[-1]

    def empty(self):
        return not self.inbox and not self.outbox
```

How the solution works

1. A stack reverses order, and two reversals restore it. That is the whole idea.
2. `push` puts the item on the inbox stack.
3. To remove from the front, the solution needs the oldest item, which sits at the bottom of the inbox. It pours the whole inbox into the outbox, which flips the order, so the oldest item ends up on top.
4. The solution pours only when the outbox is empty. Pouring at any other time would mix up the order.
5. Each item moves from the inbox to the outbox at most once, so each operation costs O(1) amortized, even though a single `pop` can cost O(n).

Time and space

Time: O(1) amortized per operation. A single `pop` can move n items, but each item moves at most once in its lifetime, so n operations cost O(n) in total.

Auxiliary space: O(n). All n items are stored in the two lists.

Memory estimate

While all 100,000 items sit in the inbox, the inbox is a list built by 100,000 appends, which `sys.getsizeof` reports as 800,984 bytes. During the first `pop`, the items pour into the outbox while the inbox still holds most of its allocation, so for a moment both lists are large, and the auxiliary memory peaks at about 1.2 MB. LeetCode limits each item to the values 1 through 9, which are shared small integers that cost nothing; this estimate assumes larger values.

Part	How it is counted	Estimate
Input	100,000 items created by the caller, × 28 bytes	2.7 MB
Auxiliary	the inbox list of 100,000 pointers (782 KB), briefly about 1.2 MB during the first `pop`	782 KB, peak 1.2 MB
Total	input plus auxiliary	3.4 MB, peak 3.9 MB

Other approaches

Approach	Time	Auxiliary space
Pour the items across on every push, so that the oldest item is always on top	O(n) per push, O(1) per pop	O(n)
Pour only when the outbox is empty (the solution above)	O(1) amortized	O(n)

Class 8: Linked lists. Problem A, the easier problem.

#206 Reverse Linked List

Easy. https://leetcode.com/problems/reverse-linked-list/

You are given `head`, the first node of a singly linked list. Reverse the list and return its new first node.

For example, the list 1 → 2 → 3 → 4 → 5 becomes 5 → 4 → 3 → 2 → 1.

Each node is a `ListNode` with the fields `val` and `next`. Start with `def reverseList(head):`

Estimate the memory for n = 5,000 nodes, which is LeetCode's limit. LeetCode's values run from −5,000 to 5,000, so count each value as a new integer.

Solution

```
def reverseList(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev
```

How the solution works

1. The solution walks the list once with two references: `cur`, the node it is fixing, and `prev`, the part already reversed, which starts empty.
2. Before it changes `cur.next`, it saves the rest of the list in `nxt`. Without this step the rest of the list would be lost.
3. It points `cur.next` backward, at `prev`.
4. It advances: `prev` becomes `cur`, and `cur` becomes `nxt`.
5. When `cur` is `None`, `prev` holds the old last node, which is the new first node.

Time and space

Time: O(n). The solution makes one pass and does a constant amount of work at each node.

Auxiliary space: O(1). It keeps three references, however long the list is.

Memory estimate

A `ListNode` costs about 90 bytes for the node and its pointer, plus 28 bytes for its value.

Part	How it is counted	Estimate
Input	5,000 nodes × (90 + 28) bytes	576 KB
Auxiliary	three references	about 150 bytes
Total	input plus auxiliary	576 KB

Other approaches

Approach	Time	Auxiliary space
Recursion: reverse the rest of the list, then attach the	O(n)	O(n) for the call stack, and it fails

Approach	Time	Auxiliary space
current node at its end		beyond Python's recursion limit of 1,000
Copy the values into a list, then write them back into the nodes in reverse order	O(n)	O(n)
Rewire the pointers in one pass (the solution above)	O(n)	O(1)

#19 Remove Nth Node From End of List

Medium. https://leetcode.com/problems/remove-nth-node-from-end-of-list/

You are given `head`, the first node of a singly linked list, and an integer n. Remove the nth node from the end of the list and return the first node of the result. When n is 1, the last node is removed. The value of n is always valid.

For example, the list 1 → 2 → 3 → 4 → 5 with n = 2 becomes 1 → 2 → 3 → 5.

Each node is a `ListNode` with the fields `val` and `next`, and `ListNode(val, next)` builds a new node. Start with `def removeNthFromEnd(head, n):`

Estimate the memory for a list of L = 30 nodes, which is LeetCode's limit. LeetCode's values run from 0 to 100, so they are shared small integers.

Solution

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head)
    fast = dummy
    slow = dummy
    for _ in range(n):
        fast = fast.next
    while fast.next:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

How the solution works

1. A singly linked list does not know its own length. Counting the nodes first takes two passes, while two pointers need only one.
2. The solution puts a dummy node in front of `head`, so that removing the first node needs no special case.
3. It moves `fast` forward n steps, which puts `fast` and `slow` n nodes apart.
4. It moves both pointers together until `fast` reaches the last node. At that point `slow` sits just before the node to delete.
5. It deletes that node with `slow.next = slow.next.next` and returns `dummy.next`, which is the first node of the result, whether or not the old first node was removed.

Time and space

Time: O(L), in one pass, where L is the length of the list. `fast` walks the whole list once, and `slow` walks L − n steps.

Auxiliary space: O(1). The solution keeps the dummy node and two references.

Memory estimate

The values are shared small integers, so each node costs about 90 bytes.

Part	How it is counted	Estimate
Input	30 nodes × 90 bytes	2.6 KB
Auxiliary	one dummy node and two references	about 250 bytes

Part	How it is counted	Estimate
Total	input plus auxiliary	2.9 KB

Other approaches

Approach	Time	Auxiliary space
Two passes: count the nodes, then walk L − n steps and remove the next node	O(L)	O(1)
Copy the nodes into a list, then remove by index	O(L)	O(L)
Two pointers n nodes apart (the solution above)	O(L), one pass	O(1)

#509 Fibonacci Number

Easy. https://leetcode.com/problems/fibonacci-number/

The Fibonacci numbers start with F(0) = 0 and F(1) = 1, and every later number is the sum of the two before it: F(n) = F(n − 1) + F(n − 2). Given n, where 0 ≤ n ≤ 30, return F(n).

For example, n = 2 gives 1, n = 3 gives 2, and n = 4 gives 3.

Start with `def fib(n):`

Estimate the memory for n = 30, which is LeetCode's limit.

Solution

```
def fib(n):
    memo = {0: 0, 1: 1}
    def go(k):
        if k not in memo:
            memo[k] = go(k - 1) + go(k - 2)
        return memo[k]
    return go(n)
```

How the solution works

1. The recursive idea is F(k) = F(k − 1) + F(k − 2), with the base cases F(0) = 0 and F(1) = 1.
2. Naive recursion calls `fib(k - 1)` and `fib(k - 2)` separately, so it computes the same values over and over. The tree of calls roughly doubles at every level, which takes $O(2^n)$ time.
3. The fix is to remember every answer in a dict, a technique called memoization. Each k is computed once and then looked up.
4. The recursion goes n levels deep before the first call returns, so the call stack holds up to n frames.
5. The time becomes O(n). The space is O(n) for the dict plus O(n) for the call stack.

Time and space

Time: O(n) with the memo, compared with $O(2^n)$ for naive recursion. Each k from 0 to n is computed once, with O(1) work.

Auxiliary space: O(n). The memo has n + 1 entries, and the recursion is n calls deep.

Memory estimate

The memo ends with 31 entries, and a dict of that size is 1,168 bytes. The values F(0) through F(13) are at most 233, so they are shared small integers; only F(14) through F(30), 17 values, are new integers. The call stack is about 30 frames deep. This small function costs about 150 bytes per frame, and `tracemalloc` does not count frames.

Part	How it is counted	Estimate
Input	one integer	28 bytes
Auxiliary	memo dict (1,168 bytes), 17 new values × 28 bytes (476 bytes), 30 frames × 150 bytes (4.4 KB)	6.0 KB
Total	input plus auxiliary	6.0 KB

Other approaches

Approach	Time	Auxiliary space
Naive recursion, with no memory of earlier answers	$O(2^n)$	O(n) for the call stack
A loop that keeps only the last two values	O(n)	O(1)

Class 9: Recursion and divide and conquer. Problem B, the harder problem.

#50 Pow(x, n)

Medium. https://leetcode.com/problems/powx-n/

Compute x raised to the power n, where x is a float and n is an integer that can be negative, zero or positive. The exponent n lies between -2^{31} and $2^{31} - 1$, so a loop that multiplies x by itself n times is far too slow. LeetCode also guarantees that $-100 < x < 100$, that x is not zero whenever n is zero or negative, and that the answer lies between -10^4 and 10^4.

For example, x = 2.0 and n = 10 give 1024.0, and x = 2.0 and n = −2 give 0.25.

Start with `def myPow(x, n):`

Estimate the memory for $n = 2^{31}$, so that the recursion is about 32 calls deep.

Solution

```
def myPow(x, n):
    if n < 0:
        return 1 / myPow(x, -n)
    if n == 0:
        return 1.0
    half = myPow(x, n // 2)
    if n % 2 == 0:
        return half * half
    return half * half * x
```

How the solution works

1. Multiplying x by itself n times takes O(n) time, and n can be about two billion.
2. The solution uses two rules instead. When n is even, x^n is $(x^{(n/2)})$ squared. When n is odd, x^n is x times (x^((n−1)/2)) squared.
3. The exponent halves at every call, so the recursion reaches the base case n = 0 after about $\log_2(n)$ calls.
4. A negative exponent is handled once, at the top: $x^{(-n)}$ is $1 / x^n$.
5. The solution computes `half = myPow(x, n // 2)` once and uses it twice. Calling `myPow` twice would undo the saving.

The code divides by zero when x is 0 and n is negative, and it returns `inf` when the answer is too large for a float. LeetCode's guarantees rule out both cases, so they need no handling at the board.

Time and space

Time: O(log n). n halves at every call, so there are about $\log_2(n)$ calls, at most 32 for a 32-bit n.

Auxiliary space: O(log n). The recursion is about $\log_2(n)$ calls deep.

Memory estimate

On the way down, each frame holds its own exponent. About 23 of the 32 exponents are larger than 256, so they are new integers. The floats appear only on the way back up, and no more than about two exist at a time. The input exponent, 2^{31}, is itself a 32-byte integer.

Part	How it is counted	Estimate
Input	one float (24 bytes) and one integer of 2^{31} (32 bytes)	56 bytes
Auxiliary	32 frames × 150 bytes, plus 23 new exponent integers × 28 bytes and a	5.4 KB

Part	How it is counted	Estimate
	few floats	
Total	input plus auxiliary	5.5 KB

Other approaches

Approach	Time	Auxiliary space
A loop that multiplies by x n times	O(n)	O(1)
Iterative fast exponentiation, which reads the bits of n in a loop	O(log n)	O(1)

#1331 Rank Transform of an Array

Easy. https://leetcode.com/problems/rank-transform-of-an-array/

You are given a list of integers, `arr`. Replace each element with its rank: the smallest value has rank 1, the next larger distinct value has rank 2, and so on, and equal values share the same rank. Return the list of ranks.

For example, `arr = [40, 10, 20, 30]` gives `[4, 1, 2, 3]`, and `arr = [37, 12, 28, 9, 100, 56, 80, 5, 12]` gives `[5, 3, 4, 2, 8, 6, 7, 1, 3]`.

Start with `def arrayRankTransform(arr):`

Estimate the memory for n = 100,000 distinct integers, all above 256.

Solution

```
def arrayRankTransform(arr):
    rank = {}
    for x in sorted(set(arr)):
        rank[x] = len(rank) + 1
    return [rank[x] for x in arr]
```

How the solution works

1. Ranks depend on order, so the solution sorts. Equal values must share a rank, so it first removes duplicates with `set(arr)`.
2. It walks through the sorted distinct values. The first gets rank 1, the next gets rank 2, and so on; `len(rank) + 1` is always the next rank.
3. It stores each value and its rank in a dict.
4. It builds the answer in the original order by looking up each element.
5. The sort dominates the time at O(n log n). The dict, the sorted list and the answer each take O(n) space.

Time and space

Time: O(n log n). Sorting the d distinct values takes O(d log d) time, with d ≤ n, and the two loops take O(n).

Auxiliary space: O(n). The set, the sorted list, the dict and the answer list each hold up to n entries.

Memory estimate

The peak comes inside the loop, when the dict grows for the last time. At that moment the sorted list (782 KB), the dict's old table (about 2.6 MB) and its new table (5.0 MB), and most of the new rank integers are all alive at once. The set is already gone by then, and the answer list does not exist yet. Every rank above 256 is a new integer, 99,744 of them.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	at the peak: sorted list (782 KB), old and new dict tables (7.6 MB), and the rank integers made so far (up to 2.7 MB)	about 11 MB
Total	input plus auxiliary	about 14 MB

When the function returns, only the answer list (782 KB) and the rank integers (2.7 MB) are left.

Other approaches

Approach	Time	Auxiliary space
Sort (value, index) pairs, then walk them once and assign ranks	O(n log n)	O(n)

#912 Sort an Array

Medium. https://leetcode.com/problems/sort-an-array/

You are given a list of integers, `nums`. Sort it in ascending order and return it. You may not use any built-in sorting function. Your solution must run in O(n log n) time and use as little extra space as possible.

For example, `nums = [5, 2, 3, 1]` gives `[1, 2, 3, 5]`, and `nums = [5, 1, 1, 2, 0, 0]` gives `[0, 0, 1, 1, 2, 5]`.

Start with `def sortArray(nums):`

Estimate the memory for n = 50,000 integers in random order, which is LeetCode's limit.

Solution

```
def sortArray(nums):
    if len(nums) <= 1:
        return nums
    mid = len(nums) // 2
    left = sortArray(nums[:mid])
    right = sortArray(nums[mid:])
    merged = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

How the solution works

1. Merge sort splits the list in half, sorts each half by the same method, and merges the two sorted halves.
2. The base case is a list of zero or one elements, which is already sorted.
3. To merge two sorted lists, the solution keeps an index into each and repeatedly takes the smaller front value. Using <= keeps equal values in their original order, which makes the sort stable.
4. When one half runs out, the solution appends what is left of the other half.
5. The recursion is $\log_2(n)$ levels deep, and at each level the merges together touch n elements, so the total work is about $n \log_2(n)$ steps.

Time and space

Time: O(n log n). There are $\log_2(n)$ levels, and each does O(n) work in merges and slices.

Auxiliary space: O(n). The merged lists and slices alive at any moment hold O(n) pointers, and the call stack adds O(log n) frames.

LeetCode asks for the smallest extra space possible. Merge sort needs O(n) extra space, while an in-place heap sort needs only O(1), so heap sort fits that wording better. This class teaches merge sort, so accept it, and use the space requirement as a question for students who finish early.

Memory estimate

The values are shared by reference, so only list slots count. At the last merge, the two sorted halves (n slots together) and the merged list (n slots) are alive at the same time, and lists built by `append` carry a few percent of spare slots. The 16 or so frames on the call stack are small and can be ignored.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	1.7 MB
Auxiliary	two sorted halves, about 8n, plus the merged list, about 8n, with a few percent of spare slots	830 KB
Total	input plus auxiliary	2.5 MB

Other approaches

Approach	Time	Auxiliary space
Insertion sort	$O(n^2)$	O(1)
Quicksort with a random pivot	O(n log n) on average, $O(n^2)$ in the worst case	O(log n)
Heap sort, done in place	O(n log n)	O(1)
The built-in `sorted()`, which is not allowed here	O(n log n)	O(n)

#1122 Relative Sort Array

Easy. https://leetcode.com/problems/relative-sort-array/

You are given two lists of integers, `arr1` and `arr2`. The values in `arr2` are distinct, and every one of them also appears in `arr1`. Sort `arr1` so that its values appear in the same relative order as in `arr2`, and put the values that do not appear in `arr2` at the end, in ascending order. Every value lies between 0 and 1,000.

For example, `arr1 = [2, 3, 1, 3, 2, 4, 6, 7, 9, 2, 19]` and `arr2 = [2, 1, 4, 3, 9, 6]` give `[2, 2, 2, 1, 4, 3, 3, 9, 6, 7, 19]`.

Start with `def relativeSortArray(arr1, arr2):`

Estimate the memory for an `arr1` of 1,000 values and an `arr2` of 500 values, which are within LeetCode's limits, with values spread evenly from 0 to 1,000, so that about three quarters of them are above 256.

Solution

```
def relativeSortArray(arr1, arr2):
    count = [0] * 1001
    for x in arr1:
        count[x] += 1
    result = []
    for x in arr2:
        result.extend([x] * count[x])
        count[x] = 0
    for x in range(1001):
        result.extend([x] * count[x])
    return result
```

How the solution works

1. The values are small, from 0 to 1,000, so the solution can count them instead of comparing them. This is counting sort: a list of 1,001 counters records how many times each value appears.
2. One pass over `arr1` fills the counters.
3. The solution then walks through `arr2` in order. For each value it appends that value as many times as it appeared, and it sets the counter to zero so the value is not written again.
4. Finally it walks through every possible value from 0 to 1,000 in ascending order and appends whatever is left. These are the values missing from `arr2`, and they come out sorted.
5. No two values are ever compared, which is how counting sort beats the O(n log n) bound that holds for sorting by comparisons.

Time and space

Time: O(n + m + k), where n is the length of `arr1`, m the length of `arr2`, and k = 1,001 the number of possible values. Counting takes n steps, the walk through `arr2` takes m steps, and the final walk takes k steps; the values written out add up to n.

Auxiliary space: O(n + k). The counters take k slots, and the result holds n values.

Memory estimate

The counters start as the shared integer 0 and stay small, so they are shared small integers. The result holds pointers to integers that already exist, except for the leftover values above 256, which the final loop creates afresh; there are about a hundred of them here.

Part	How it is counted	Estimate
Input	`arr1`, 56 + 8 × 1,000 plus about 750 integers × 28 bytes; `arr2`, 56 + 8 × 500 plus about 370 integers × 28 bytes	42 KB
Auxiliary	the counters, 56 + 8 × 1,001 (7.9 KB); the result, 56 + 8 × 1,000 (7.9 KB); about 100 new integers × 28 bytes (2.7 KB)	19 KB
Total	input plus auxiliary	61 KB

Other approaches

Approach	Time	Auxiliary space
Sort `arr1` with a key: a value's position in `arr2`, or the value itself after every position	O(n log n + m)	O(n + m)
Count with a dict, then sort only the leftover values	O(n + m + d log d), where d is the number of leftover distinct values	O(n)

#75 Sort Colors

Medium. https://leetcode.com/problems/sort-colors/

You are given a list, `nums`, of n objects colored red, white or blue, written as 0, 1 and 2. Sort the list in place so that objects of the same color are next to each other, with the colors in the order red, white, blue. You may not use a library sort. LeetCode asks for a single pass that uses only constant extra space.

For example, `nums = [2, 0, 2, 1, 1, 0]` becomes `[0, 0, 1, 1, 2, 2]`, and `nums = [2, 0, 1]` becomes `[0, 1, 2]`.

The function returns nothing; it changes `nums` itself. Start with `def sortColors(nums):`

Estimate the memory for n = 300 values, which is LeetCode's limit.

Solution

```
def sortColors(nums):
    low = 0
    mid = 0
    high = len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
```

How the solution works

1. This is the partition step of quicksort, done with three groups instead of two. Edsger Dijkstra posed it as the Dutch national flag problem.
2. Three indexes split the list into four regions. Everything before `low` is 0, everything from `low` up to `mid` is 1, everything after `high` is 2, and the region from `mid` to `high` has not been looked at yet.
3. When `nums[mid]` is 0, the solution swaps it into the 0 region and moves both `low` and `mid` forward.
4. When it is 1, it is already in the right region, so only `mid` moves forward. When it is 2, the solution swaps it into the 2 region and moves `high` back, but leaves `mid` where it is, because the value just swapped in has not been looked at.
5. The loop ends when the unexamined region is empty.

Time and space

Time: O(n), in one pass. Each step either moves `mid` forward or moves `high` back, so the loop runs at most n times.

Auxiliary space: O(1). The solution keeps three indexes.

Memory estimate

The values 0, 1 and 2 are shared small integers, so the input costs only its pointers, and the swaps only move pointers around.

Part	How it is counted	Estimate
Input	56 + 8 × 300	2.4 KB
Auxiliary	three indexes	about 100 bytes
Total	input plus auxiliary	2.5 KB

Other approaches

Approach	Time	Auxiliary space
Count the 0s, 1s and 2s, then overwrite the list: counting sort in two passes	O(n)	O(1)
A library sort, which the problem forbids	O(n log n)	O(n) in Python

Class 12: Binary search. Problem A, the easier problem.

#704 Binary Search

Easy. https://leetcode.com/problems/binary-search/

You are given a list of distinct integers, `nums`, sorted in ascending order, and an integer, `target`. Return the index of `target`, or −1 if it is not in the list. Your solution must run in O(log n) time.

For example, with `nums = [-1, 0, 3, 5, 9, 12]`, `target = 9` gives 4 and `target = 2` gives −1.

Start with `def search(nums, target):`

Estimate the memory for n = 10,000 sorted distinct integers, which is LeetCode's limit, none of them between −5 and 256.

Solution

```
def search(nums, target):
    lo = 0
    hi = len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

How the solution works

1. The invariant is this: if `target` is in the list, it lies somewhere from `nums[lo]` to `nums[hi]`, both ends included.
2. The solution looks at the middle value. If it equals `target`, the search is over.
3. If the middle value is smaller than `target`, the target must lie to the right, so `lo = mid + 1`. Otherwise it lies to the left, so `hi = mid - 1`.
4. Each step throws away at least half of the remaining range, so there are at most about $\log_2(n) + 1$ steps.
5. When `lo > hi` the range is empty, so `target` is not in the list, and the function returns −1.

Time and space

Time: O(log n). The range halves at every step, n, n/2, n/4 and so on down to 1, so the loop runs about $\log_2(n)$ times.

Auxiliary space: O(1). The solution keeps `lo`, `hi` and `mid`.

Memory estimate

Part	How it is counted	Estimate
Input	56 + 8n + 28n	352 KB
Auxiliary	`lo`, `hi` and `mid`	about 100 bytes
Total	input plus auxiliary	352 KB

Other approaches

Approach	Time	Auxiliary space
A linear scan	O(n)	O(1)
`bisect_left` from the standard library, which is not allowed at the board	O(log n)	O(1)

Class 12: Binary search. Problem B, the harder problem.

#34 Find First and Last Position of Element in Sorted Array

Medium. https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/

You are given a list of integers, `nums`, sorted in non-decreasing order, so it may contain repeated values, and an integer, `target`. Return `[first, last]`, the first and the last index at which `target` appears, or `[-1, -1]` if it does not appear. Your solution must run in O(log n) time.

For example, with `nums = [5, 7, 7, 8, 8, 10]`, `target = 8` gives `[3, 4]` and `target = 6` gives `[-1, -1]`.

Start with `def searchRange(nums, target):`

Estimate the memory for n = 100,000 sorted integers above 256, which is LeetCode's limit, including a block of 1,000 copies of `target`.

Solution

```
def searchRange(nums, target):
    def first_at_least(x):
        lo = 0
        hi = len(nums)
        while lo < hi:
            mid = (lo + hi) // 2
            if nums[mid] < x:
                lo = mid + 1
            else:
                hi = mid
        return lo
    start = first_at_least(target)
    if start == len(nums) or nums[start] != target:
        return [-1, -1]
    return [start, first_at_least(target + 1) - 1]
```

How the solution works

1. A plain binary search finds some copy of `target`, but not necessarily the first or the last one. Walking left or right from that copy could take O(n) time.
2. The helper `first_at_least(x)` returns the index of the first element that is at least x, or n if there is none. It is a binary search that does not stop when it finds x: when `nums[mid] >= x`, it keeps `mid` in the range with `hi = mid`.
3. The first position of `target` is `first_at_least(target)`. If that index is past the end, or the value there is not `target`, the answer is `[-1, -1]`.
4. The last position of `target` is `first_at_least(target + 1) - 1`, because the values are integers, so `target + 1` is the first value larger than `target`.
5. Two binary searches take O(log n) time and O(1) space.

Time and space

Time: O(log n). The solution runs two binary searches of about $\log_2(n)$ steps each.

Auxiliary space: O(1). It keeps a few indexes and the two-element result list.

Memory estimate

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB

Part	How it is counted	Estimate
Auxiliary	the result list (72 bytes), two indexes and `lo`, `hi` and `mid`	about 250 bytes
Total	input plus auxiliary	3.4 MB

Other approaches

Approach	Time	Auxiliary space
A linear scan	O(n)	O(1)
`bisect_left` and `bisect_right`, which are not allowed at the board	O(log n)	O(1)

#104 Maximum Depth of Binary Tree

Easy. https://leetcode.com/problems/maximum-depth-of-binary-tree/

You are given `root`, the root node of a binary tree. Return the tree's maximum depth, which is the number of nodes along the longest path from the root down to a leaf.

For example, `root = [3, 9, 20, null, null, 15, 7]` gives 3, and `root = [1, null, 2]` gives 2. LeetCode writes a tree as a list of its values level by level, with `null` for a missing child.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def maxDepth(root):`

Estimate the memory for n = 10,000 nodes in a balanced tree, which is 14 levels tall. LeetCode's values run from −100 to 100, so most of them are shared small integers.

Solution

```
def maxDepth(root):
    if root is None:
        return 0
    left = maxDepth(root.left)
    right = maxDepth(root.right)
    return 1 + max(left, right)
```

How the solution works

1. A tree is made of smaller trees: a root, a left subtree and a right subtree. That shape suggests recursion.
2. The base case is an empty tree, which has depth 0.
3. Otherwise the depth is 1, for this node, plus the larger of the two subtree depths.
4. The solution visits every node exactly once, so it takes O(n) time.
5. Its space is the depth of the recursion, which equals the height h of the tree: O(log n) when the tree is balanced and O(n) when the tree is a single chain.

In plain Python the recursion limit is 1,000 calls, so this solution raises `RecursionError` on a chain of 1,000 or more nodes, and LeetCode allows trees of up to 10,000 nodes. Accept the recursive solution, and ask the student what happens on a long chain. The iterative breadth-first search under "Other approaches" avoids the limit.

Time and space

Time: O(n). The solution makes one call for each node.

Auxiliary space: O(h), where h is the height of the tree. The call stack is as deep as the tree is tall, and a balanced tree has a height of about $\log_2(n)$.

Memory estimate

A `TreeNode` costs about 100 bytes for the node and its two pointers.

Part	How it is counted	Estimate
Input	10,000 nodes × 100 bytes	977 KB
Auxiliary	14 frames × 150 bytes	2.1 KB
Total	input plus auxiliary	979 KB

Other approaches

Approach	Time	Auxiliary space
Breadth-first search with a queue, counting the levels	O(n)	O(w), where w is the width of the widest level
Depth-first search with an explicit stack of (node, depth) pairs	O(n)	O(h)

#102 Binary Tree Level Order Traversal

Medium. https://leetcode.com/problems/binary-tree-level-order-traversal/

You are given `root`, the root node of a binary tree. Return its level order traversal: a list of lists, in which list i holds the values of all the nodes at depth i, from left to right.

For example, `root = [3, 9, 20, null, null, 15, 7]` gives `[[3], [9, 20], [15, 7]]`.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def levelOrder(root):`

Estimate the memory for n = 10,000 nodes in a complete tree, which is 14 levels tall. That is five times LeetCode's limit of 2,000 nodes. LeetCode's values run from −1,000 to 1,000, so count each value as a new integer.

Solution

```
from collections import deque

def levelOrder(root):
    if root is None:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

How the solution works

1. Recursion goes deep first, but this problem wants one level at a time. That is breadth-first search, and a queue, first in and first out, provides it.
2. The solution starts with the root in a deque, which needs the import on the first line.
3. At the start of each pass, `len(queue)` is exactly the number of nodes on the current level, so the solution processes that many nodes.
4. For each node it removes the node from the front, records its value, and adds its children at the back, where they form the next level.
5. When the queue is empty, every level has been recorded.

Time and space

Time: O(n). Each node enters the queue once and leaves it once.

Auxiliary space: O(n). The queue holds at most about one level, which can be about n/2 nodes in a full tree, and the result stores all n values.

Memory estimate

A `TreeNode` costs about 100 bytes plus 28 bytes for its value. The queue is widest, at about 5,000 entries, while the last full level is being processed, and at that moment the result holds only about half of the values. The queue and the full result are never at their largest together.

Part	How it is counted	Estimate
Input	10,000 nodes × (100 + 28) bytes	1.2 MB
Auxiliary	at the peak: a queue of about 5,000 entries (42 KB) and level lists holding about half the values (about 45 KB)	about 90 KB
Total	input plus auxiliary	1.3 MB

Other approaches

Approach	Time	Auxiliary space
Recursive depth-first search that passes the depth and appends to `result[depth]`	O(n)	O(h) plus the result

#700 Search in a Binary Search Tree

Easy. https://leetcode.com/problems/search-in-a-binary-search-tree/

You are given `root`, the root node of a binary search tree, and a value, `val`. Return the node whose value equals `val`, or `None` if there is no such node. The node you return stands for the whole subtree below it.

For example, with `root = [4, 2, 7, 1, 3]`, `val = 2` gives the subtree `[2, 1, 3]`, and `val = 5` gives `None`.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def searchBST(root, val):`

Estimate the memory for n = 5,000 nodes, which is LeetCode's limit, in a balanced tree. LeetCode's values run from 1 to 10^7, so count each value as a new integer.

Solution

```
def searchBST(root, val):
    node = root
    while node is not None and node.val != val:
        if val < node.val:
            node = node.left
        else:
            node = node.right
    return node
```

How the solution works

1. In a binary search tree, every value to the left of a node is smaller than the node's value, and every value to the right is larger.
2. One comparison therefore tells the solution which whole side of the tree to ignore, just as in binary search.
3. The solution starts at the root. If `val` is smaller than the node's value it goes left; otherwise it goes right.
4. It stops when it finds `val` or falls off the tree, when `node` becomes `None`, and it returns `node`.
5. The search follows a single path from the root toward a leaf, so it takes O(h) time and, written as a loop, O(1) space. A recursive version would use O(h) space for the call stack.

Time and space

Time: O(h), where h is the height of the tree. The solution takes at most one step per level: about $\log_2(n)$ steps when the tree is balanced, and n when the tree is a chain.

Auxiliary space: O(1). The solution keeps a single reference.

Memory estimate

A `TreeNode` costs about 100 bytes for the node and its two pointers, plus 28 bytes for its value.

Part	How it is counted	Estimate
Input	5,000 nodes × (100 + 28) bytes	625 KB
Auxiliary	one reference	about 100 bytes
Total	input plus auxiliary	625 KB

Other approaches

Approach	Time	Auxiliary space
The same search written recursively	O(h)	O(h) for the call stack
Visit every node, ignoring the search-tree property	O(n)	O(h)
Follow one path down the tree in a loop (the solution above)	O(h)	O(1)

#98 Validate Binary Search Tree

Medium. https://leetcode.com/problems/validate-binary-search-tree/

You are given `root`, the root node of a binary tree. Return `True` if it is a valid binary search tree, and `False` otherwise. A tree is valid when, for every node, all the values in its left subtree are strictly smaller than the node's value and all the values in its right subtree are strictly larger.

For example, `root = [2, 1, 3]` gives `True`, and `root = [5, 1, 4, null, null, 3, 6]` gives `False`, because the root's right child, 4, is smaller than 5.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def isValidBST(root):`

Estimate the memory for n = 10,000 nodes, which is LeetCode's limit, in a balanced valid tree, 14 levels tall. LeetCode's values can be as large as 2^{31} in size, so count each value as a new integer.

Solution

```
def isValidBST(root):
    def ok(node, low, high):
        if node is None:
            return True
        if not (low < node.val < high):
            return False
        return ok(node.left, low, node.val) and ok(node.right, node.val, high)
    return ok(root, float('-inf'), float('inf'))
```

How the solution works

1. A common mistake is to compare each node only with its own children. In the tree `[5, 4, 6, null, null, 3, 7]`, every parent and child are in the right order, but 3 sits in the right subtree of 5 and is smaller than 5, so the tree is not valid. A check of children alone would wrongly return `True`.
2. Every node must instead lie inside a range, from `low` to `high`, set by all of its ancestors. The root may hold any value, so its range runs from minus infinity to plus infinity.
3. Going left, the current value becomes the new upper limit: `ok(node.left, low, node.val)`.
4. Going right, the current value becomes the new lower limit: `ok(node.right, node.val, high)`.
5. If any node falls outside its range, the answer is `False`. The solution visits each node at most once.

In plain Python the recursion limit is 1,000 calls, so this solution raises `RecursionError` on a chain of 1,000 or more nodes, and LeetCode allows trees of up to 10,000 nodes. Accept the recursive solution, and ask the student what happens on a long chain. The iterative in-order traversal under "Other approaches" avoids the limit.

Time and space

Time: O(n). The solution checks each node once.

Auxiliary space: O(h), where h is the height of the tree. The recursion is as deep as the tree is tall.

Memory estimate

Each call holds four values, and a frame costs about 150 bytes. The two infinities are floats of 24 bytes each.

Part	How it is counted	Estimate
Input	10,000 nodes × (100 + 28) bytes	1.2 MB
Auxiliary	14 frames × 150 bytes, plus two floats	2.1 KB

Part	How it is counted	Estimate
Total	input plus auxiliary	1.2 MB

Other approaches

Approach	Time	Auxiliary space
In-order traversal with an explicit stack, checking that each value is larger than the one before	O(n)	O(h)
Copy the in-order sequence into a list, then check that it is strictly increasing	O(n)	O(n)
Pass a range down the recursion (the solution above)	O(n)	O(h)

#110 Balanced Binary Tree

Easy. https://leetcode.com/problems/balanced-binary-tree/

You are given `root`, the root node of a binary tree. Return `True` if the tree is height-balanced and `False` otherwise. A tree is height-balanced when, at every node, the heights of the left and right subtrees differ by at most one.

For example, `root = [3, 9, 20, null, null, 15, 7]` gives `True`, `root = [1, 2, 2, 3, 3, null, null, 4, 4]` gives `False`, and `root = []` gives `True`.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def isBalanced(root):`

Estimate the memory for n = 5,000 nodes, which is LeetCode's limit, in a balanced tree, 13 levels tall. LeetCode's values run from −10,000 to 10,000, so count each value as a new integer.

Solution

```
def isBalanced(root):
    def height(node):
        if node is None:
            return 0
        left = height(node.left)
        if left == -1:
            return -1
        right = height(node.right)
        if right == -1 or abs(left - right) > 1:
            return -1
        return 1 + max(left, right)
    return height(root) != -1
```

How the solution works

1. The obvious approach computes the height of both subtrees at every node, which recomputes the same heights again and again.
2. The solution computes each height once instead, from the bottom up. `height(node)` returns the height of the subtree, or −1 if that subtree is already unbalanced.
3. The base case is an empty tree, whose height is 0.
4. At each node the solution gets the left height and then the right height. If either is −1, or they differ by more than one, it passes −1 up the tree, and the search stops early.
5. The tree is balanced exactly when the height of the root is not −1.

The balanced trees that this class studies, such as AVL trees, keep this property after every insertion and deletion by rotating nodes, which is what guarantees O(log n) search.

In plain Python the recursion limit is 1,000 calls, so this solution raises `RecursionError` on a chain of 1,000 or more nodes, and LeetCode allows 5,000. A chain is unbalanced, but the solution walks to its bottom before it can say so. Accept the recursive solution, and ask the student what happens on a long chain.

Time and space

Time: O(n). The solution visits each node once.

Auxiliary space: O(h), where h is the height of the tree. The recursion is as deep as the tree is tall.

Memory estimate

A `TreeNode` costs about 100 bytes plus 28 bytes for its value. The recursion is 13 calls deep on this tree.

Part	How it is counted	Estimate
Input	5,000 nodes × (100 + 28) bytes	625 KB
Auxiliary	13 frames × 150 bytes	2.0 KB
Total	input plus auxiliary	627 KB

Other approaches

Approach	Time	Auxiliary space
At every node, compute both subtree heights from scratch	$O(n^2)$ in the worst case, O(n log n) on a balanced tree	O(h)
The same bottom-up check with an explicit stack instead of recursion	O(n)	O(n)

#1382 Balance a Binary Search Tree

Medium. https://leetcode.com/problems/balance-a-binary-search-tree/

You are given `root`, the root node of a binary search tree. Return a balanced binary search tree with the same values. A binary search tree is balanced when, at every node, the depths of the two subtrees differ by at most one. If more than one answer is possible, return any of them.

For example, `root = [1, null, 2, null, 3, null, 4]`, a chain, can give `[2, 1, 3, null, null, null, 4]`; `[3, 1, 4, null, 2]` is also correct. And `root = [2, 1, 3]` gives `[2, 1, 3]`.

Each node is a `TreeNode` with the fields `val`, `left` and `right`. Start with `def balanceBST(root):`

Estimate the memory for n = 10,000 nodes, which is LeetCode's limit, arranged in a single chain to the right, as in the first example. LeetCode's values run from 1 to 100,000, so count each value as a new integer.

Solution

```
def balanceBST(root):
    nodes = []
    stack = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        nodes.append(node)
        node = node.right
    def build(lo, hi):
        if lo > hi:
            return None
        mid = (lo + hi) // 2
        top = nodes[mid]
        top.left = build(lo, mid - 1)
        top.right = build(mid + 1, hi)
        return top
    return build(0, len(nodes) - 1)
```

How the solution works

1. An in-order traversal of a binary search tree visits its values in sorted order. The solution collects the nodes in that order.
2. It traverses with an explicit stack rather than with recursion, because the input can be a chain 10,000 nodes long, far deeper than Python's recursion limit of 1,000.
3. To build a balanced tree from a sorted list, it makes the middle node the root, builds the left half into the left subtree, and builds the right half into the right subtree.
4. Because each half differs in size by at most one, every subtree's two sides differ in depth by at most one, so the new tree is balanced.
5. The solution reuses the existing nodes and only changes their `left` and `right` pointers, so it creates no new nodes. The recursion in `build` is only about $\log_2(n)$ calls deep, which is safe.

An AVL tree keeps itself balanced as values arrive, by rotating nodes after each insertion. This solution balances the tree all at once instead, which is simpler when the whole tree is available.

Time and space

Time: O(n). The traversal visits each node once, and `build` places each node once.

Auxiliary space: O(n). The list holds a pointer to every node, the stack holds up to h nodes during the traversal, and the recursion in `build` is O(log n) deep.

Memory estimate

A `TreeNode` costs about 100 bytes plus 28 bytes for its value. On a chain to the right, the stack never holds more than one node. The list of 10,000 nodes, built with `append`, is 85,176 bytes.

Part	How it is counted	Estimate
Input	10,000 nodes × (100 + 28) bytes	1.2 MB
Auxiliary	the list of 10,000 node pointers (83 KB), a stack of one node, and 14 frames of `build` × 150 bytes	85 KB
Total	input plus auxiliary	1.3 MB

Other approaches

Approach	Time	Auxiliary space
Collect the values instead, and build a tree of new nodes	O(n)	O(n), plus n new nodes
Insert every value into an AVL tree	O(n log n)	O(n)
The Day-Stout-Warren algorithm, which straightens the tree into a chain and then balances it by rotations	O(n)	O(1)

Class 16: Heaps, priority queues and heap sort. Problem A, the easier problem.

#703 Kth Largest Element in a Stream

Easy. https://leetcode.com/problems/kth-largest-element-in-a-stream/

Design a class `KthLargest`. It is created with an integer, `k`, and an initial list of integers, `nums`. Its method `add(val)` adds `val` to the stream of values and returns the kth largest value of everything seen so far, counting repeated values separately.

For example, after `KthLargest(3, [4, 5, 8, 2])`, the calls `add(3)`, `add(5)`, `add(10)`, `add(9)` and `add(4)` return 4, 5, 5, 8 and 8.

Start with `class KthLargest:` and write the methods `__init__(self, k, nums)` and `add(self, val)`.

Estimate the memory for k = 10,000, an initial list of 10,000 values and 10,000 calls to `add`, which are LeetCode's limits. The values lie between −10,000 and 10,000, and the caller creates each one.

Solution

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = []
        for x in nums:
            self.add(x)

    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]
```

How the solution works

1. Sorting the whole stream on every call to `add` costs O(n log n) per call.
2. Only the k largest values matter, so the solution keeps them in a min-heap of size k. The smallest of the k largest values sits at the top, in `heap[0]`.
3. That top value is exactly the kth largest value of everything seen so far.
4. `add(val)` pushes `val`. If the heap now holds more than k values, it pops the smallest, and the heap again holds the k largest.
5. A push or a pop takes O(log k) time, and looking at `heap[0]` takes O(1).

Time and space

Time: O(log k) per call to `add`. Each call does one push and at most one pop on a heap of at most k + 1 values.

Auxiliary space: O(k). The heap never holds more than k + 1 values.

Memory estimate

The heap holds pointers to integers that the caller created, so it adds no new integers. A list built by 10,001 pushes is 85,176 bytes.

Part	How it is counted	Estimate
Input	the initial list, 56 + 8 × 10,000 (78 KB), plus 20,000 integers × 28 bytes	625 KB

Part	How it is counted	Estimate
	(547 KB)	
Auxiliary	the heap list of up to 10,001 pointers	83 KB
Total	input plus auxiliary	708 KB

Other approaches

Approach	Time	Auxiliary space
Sort the whole stream on every call	O(n log n) per call	O(n)
Keep a sorted list and insert each value at its place found by binary search	O(n) per call, because the insertion shifts the list	O(n)
A min-heap of size k (the solution above)	O(log k) per call	O(k)

#347 Top K Frequent Elements

Medium. https://leetcode.com/problems/top-k-frequent-elements/

You are given a list of integers, `nums`, and an integer, `k`. Return the k values that appear most often, in any order. The answer is guaranteed to be unique, and your solution should be faster than O(n log n).

For example, `nums = [1, 1, 1, 2, 2, 3]` and `k = 2` give `[1, 2]`.

Start with `def topKFrequent(nums, k):`

Estimate the memory for n = 100,000 values, which is LeetCode's limit, made of 20,000 distinct values between −10,000 and 10,000 that each appear five times, and k = 100. Assume that the copies of a value are the same integer object.

Solution

```
import heapq

def topKFrequent(nums, k):
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    heap = []
    for x, c in counts.items():
        heapq.heappush(heap, (c, x))
        if len(heap) > k:
            heapq.heappop(heap)
    return [x for c, x in heap]
```

How the solution works

1. First the solution counts every value in a dict that maps a value to its frequency, which takes O(n) time.
2. It then needs the k largest counts. Sorting all d distinct values would cost O(d log d).
3. Instead it keeps a min-heap of (count, value) pairs that never holds more than k pairs, with the smallest count at the top.
4. For each pair it pushes the pair, and if the heap now holds more than k pairs, it pops the one with the smallest count. The heap therefore keeps the k most frequent values.
5. At the end the heap holds the answer, and the solution reads the values out.

Time and space

Time: O(n + d log k), where d is the number of distinct values. Counting takes O(n), and each of the d distinct values costs one push and at most one pop on a heap of size k + 1.

Auxiliary space: O(d + k). The dict holds d entries and the heap holds k.

Memory estimate

The counts are at most a handful, so they are shared small integers. A dict of 20,000 integer keys is 589,912 bytes, and the last time it grows it briefly holds its old and its new table, about 1.5 times its final size. The heap holds 100 tuples of 56 bytes each, and both the heap list and the result list are about 920 bytes.

Part	How it is counted	Estimate
Input	56 + 8n, plus 20,000 distinct integers × 28 bytes	1.3 MB
Auxiliary	the dict (576 KB, peaking near 864 KB while it grows), the heap (6.4	584 KB, peak

Part	How it is counted	Estimate
	KB) and the result list (0.9 KB)	about 870 KB
Total	input plus auxiliary	1.9 MB, peak about 2.2 MB

Other approaches

Approach	Time	Auxiliary space
Sort the (count, value) pairs	O(n + d log d)	O(d)
Bucket sort by frequency: a list of buckets indexed by count	O(n)	O(n)

#733 Flood Fill

Easy. https://leetcode.com/problems/flood-fill/

`image` is a grid of integers, a picture of pixel colors. Starting from the pixel at row `sr` and column `sc`, change to `color` that pixel and every pixel connected to it by moves up, down, left or right through pixels of the same starting color. Return the changed image.

For example, `image = [[1, 1, 1], [1, 1, 0], [1, 0, 1]]`, `sr = 1`, `sc = 1` and `color = 2` give `[[2, 2, 2], [2, 2, 0], [2, 0, 1]]`.

Start with `def floodFill(image, sr, sc, color):`

Estimate the memory for a 50 × 50 grid, which is LeetCode's limit, so n = 2,500 cells, all of one color. This is the worst case.

Solution

```
def floodFill(image, sr, sc, color):
    old = image[sr][sc]
    if old == color:
        return image
    rows = len(image)
    cols = len(image[0])
    image[sr][sc] = color
    stack = [(sr, sc)]
    while stack:
        r, c = stack.pop()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and image[nr][nc] == old:
                image[nr][nc] = color
                stack.append((nr, nc))
    return image
```

How the solution works

1. The grid is a graph: each cell is a vertex, and its four neighbors are its edges. The task is to reach every vertex connected through cells of the old color, which is a depth-first search.
2. If the old color equals the new one, nothing would change and the search could loop forever, so the solution returns at once.
3. The solution recolors a cell at the moment it pushes the cell. The new color then means "already visited", so no separate visited set is needed.
4. It pops a cell and looks at its four neighbors, pushing each one that lies inside the grid and still has the old color.
5. An explicit stack avoids Python's recursion limit on large regions. Each cell is pushed at most once.

Time and space

Time: O(m × n). Each cell is pushed and popped at most once and checks four neighbors.

Auxiliary space: O(m × n). In the worst case the stack holds a large share of the cells.

Memory estimate

All the values are shared small integers, so the grid costs only its list slots. Each stack entry is a tuple of 56 bytes plus an 8-byte slot, and the row and column numbers are below 257, so they are shared too. Starting from a corner, the stack holds at most about half of the cells.

Part	How it is counted	Estimate
Input	50 lists × (56 + 8 × 50), plus the outer list, 56 + 8 × 50	23 KB
Auxiliary	about 1,250 cells on the stack, each a 56-byte tuple plus an 8-byte slot	78 KB
Total	input plus auxiliary	101 KB

Other approaches

Approach	Time	Auxiliary space
Recursive depth-first search	O(m × n)	O(m × n) for the call stack in the worst case
Breadth-first search with a deque	O(m × n)	between O(m + n) and O(m × n)

#200 Number of Islands

Medium. https://leetcode.com/problems/number-of-islands/

You are given a grid of the characters `"1"`, for land, and `"0"`, for water. Count the islands. An island is a group of land cells connected horizontally or vertically, and the grid is surrounded by water on every side.

For example, `grid = [["1","1","0","0","0"], ["1","1","0","0","0"], ["0","0","1","0","0"], ["0","0","0","1","1"]]` gives 3.

Start with `def numIslands(grid):`

Estimate the memory for a 300 × 300 grid, which is LeetCode's limit, so n = 90,000 cells, all land, which makes one island.

Solution

```
def numIslands(grid):
    rows = len(grid)
    cols = len(grid[0])
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                grid[r][c] = '0'
                stack = [(r, c)]
                while stack:
                    cr, cc = stack.pop()
                    for nr, nc in ((cr + 1, cc), (cr - 1, cc), (cr, cc + 1), (cr,
cc - 1)):
                        if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] ==
'1':
                            grid[nr][nc] = '0'
                            stack.append((nr, nc))
    return count
```

How the solution works

1. The solution scans every cell. The first time it meets a land cell, it has found a new island, so it counts it.
2. It then erases the whole island so that it never counts it again: a depth-first search from that cell turns every connected `"1"` into `"0"`, and the sunk cell doubles as the visited mark.
3. It marks a cell when it pushes the cell, not when it pops it, so no cell is pushed twice.
4. The outer scan visits each cell once, and the searches together visit each land cell once.
5. Every cell is therefore handled a constant number of times.

Time and space

Time: O(m × n). The scan covers all m × n cells, and each land cell is pushed and popped once.

Auxiliary space: O(m × n). In the worst case, one large island, the stack can hold a large share of the cells.

Memory estimate

The grid holds one-character strings, which are shared, so only its list slots count. Each stack entry is a tuple of 56 bytes plus an 8-byte slot, and the row and column numbers above 256 add a few new integers. At its peak the stack holds about half of the cells.

Part	How it is counted	Estimate
Input	300 lists × (56 + 8 × 300), plus the outer list	722 KB
Auxiliary	about 45,000 cells on the stack, each about 72 bytes with its tuple, slot and share of new integers	3.1 MB
Total	input plus auxiliary	3.8 MB

Other approaches

Approach	Time	Auxiliary space
Recursive depth-first search	O(m × n)	O(m × n) for the call stack
Breadth-first search with a deque	O(m × n)	usually about O(min(m, n)) for the queue
Union-find, from Class 22	about O(m × n)	O(m × n)

#1971 Find if Path Exists in Graph

Easy. https://leetcode.com/problems/find-if-path-exists-in-graph/

A graph has n vertices, numbered 0 to n − 1, and a list of undirected edges. Return `True` if there is a path from `source` to `destination`, and `False` otherwise.

For example, `n = 3`, `edges = [[0, 1], [1, 2], [2, 0]]`, `source = 0` and `destination = 2` give `True`, while `n = 6`, `edges = [[0, 1], [0, 2], [3, 5], [5, 4], [4, 3]]`, `source = 0` and `destination = 5` give `False`.

Start with `def validPath(n, edges, source, destination):`

Estimate the memory for n = 100,000 vertices joined in a single long path, with edges [i, i + 1]. LeetCode's limit is 200,000 vertices.

Solution

```
from collections import deque

def validPath(n, edges, source, destination):
    graph = {i: [] for i in range(n)}
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
    seen = {source}
    queue = deque([source])
    while queue:
        node = queue.popleft()
        if node == destination:
            return True
        for nxt in graph[node]:
            if nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    return False
```

How the solution works

1. A list of edges is awkward to search, so the solution first builds an adjacency list: a dict from each vertex to the list of its neighbors. The edges are undirected, so it adds each edge in both directions.
2. It then runs a breadth-first search from `source`. A queue holds the vertices still to explore, and a set remembers the vertices already reached.
3. It takes a vertex from the queue. If that vertex is `destination`, a path exists.
4. Otherwise it adds every unseen neighbor to the set and to the queue.
5. If the queue empties before the search reaches `destination`, the destination lies in a different part of the graph, and the answer is `False`.

Time and space

Time: O(V + E). Building the graph takes O(V + E), and the search takes each vertex out of the queue once and looks at each edge twice.

Auxiliary space: O(V + E). The adjacency list holds V lists with 2E entries in total, and the set and the queue hold at most V vertices.

Memory estimate

Each edge in the input is a list of two integers, 72 bytes plus two integers of 28 bytes. The graph dict costs about 52 bytes per entry at this size, its key integers from `range` are new, and each vertex's neighbor list, built by `append`, is 88 bytes. The set costs about 42 bytes per vertex. On a path the queue never holds more than a vertex or two.

Part	How it is counted	Estimate
Input	100,000 edges × (72 + 2 × 28) bytes, plus the outer list, 56 + 8n	13 MB
Auxiliary	the graph, 100,000 × (52 + 28 + 88) bytes, plus the set, 42 bytes per vertex; the queue is tiny	20 MB
Total	input plus auxiliary	33 MB

Other approaches

Approach	Time	Auxiliary space
Depth-first search with a stack	O(V + E)	O(V + E)
Union-find, from Class 22	about O(V + E)	O(V)

#994 Rotting Oranges

Medium. https://leetcode.com/problems/rotting-oranges/

A grid holds 0 for an empty cell, 1 for a fresh orange and 2 for a rotten orange. Every minute, each fresh orange next to a rotten orange, up, down, left or right, becomes rotten. Return the smallest number of minutes after which no fresh orange remains, or −1 if that can never happen.

For example, `grid = [[2, 1, 1], [1, 1, 0], [0, 1, 1]]` gives 4, `grid = [[2, 1, 1], [0, 1, 1], [1, 0, 1]]` gives −1, and `grid = [[0, 2]]` gives 0.

Start with `def orangesRotting(grid):`

Estimate the memory for a 10 × 10 grid, which is LeetCode's limit, so n = 100 cells, with a single rotten orange in a corner and every other cell fresh.

Solution

```
from collections import deque

def orangesRotting(grid):
    rows = len(grid)
    cols = len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    while queue and fresh:
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr, nc))
        minutes += 1
    return minutes if fresh == 0 else -1
```

How the solution works

1. All the rotten oranges spread at the same time. That is a breadth-first search that starts with every rotten orange in the queue at once, called a multi-source search.
2. One level of the search equals one minute. The solution processes exactly `len(queue)` cells per minute, as the level-order traversal of Class 13 did.
3. It counts the fresh oranges first. Each time one rots, it subtracts 1 and adds that orange to the queue.
4. It stops when the queue is empty or no fresh orange is left. The number of levels processed is the answer.
5. If a fresh orange is left at the end, the rot never reached it, and the answer is −1.

Time and space

Time: O(m × n). The solution scans every cell once, and each orange enters the queue at most once.

Auxiliary space: O(m × n). The queue can hold many cells at once.

Memory estimate

The queue is a deque of about 760 bytes for its first block, holding (row, column) tuples of 56 bytes each. At most about 10 cells share a level of the search here.

Part	How it is counted	Estimate
Input	10 lists × (56 + 80), plus the outer list	1.5 KB
Auxiliary	the deque (760 bytes), about 10 live tuples × 64 bytes, and tuples made and freed along the way	about 2 KB
Total	input plus auxiliary	3.5 KB

Other approaches

Approach	Time	Auxiliary space
Simulate each minute by rescanning the whole grid	$O((m \times n)^2)$	O(1)

Class 19: Topological sort. Problem A, the easier problem.

#207 Course Schedule

Medium. https://leetcode.com/problems/course-schedule/

There are `numCourses` courses, numbered 0 to `numCourses − 1`. Each entry `prerequisites[i] = [a, b]` means that you must take course b before course a. Return `True` if you can finish every course, which is possible exactly when the prerequisites contain no cycle, and `False` otherwise.

For example, `numCourses = 2` and `prerequisites = [[1, 0]]` give `True`, while `numCourses = 2` and `prerequisites = [[1, 0], [0, 1]]` give `False`.

Start with `def canFinish(numCourses, prerequisites):`

Estimate the memory for V = 2,000 courses, which is LeetCode's limit, in one long chain of prerequisites, so E = 1,999.

Solution

```
from collections import deque

def canFinish(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    indegree = [0] * numCourses
    for course, pre in prerequisites:
        graph[pre].append(course)
        indegree[course] += 1
    queue = deque(i for i in range(numCourses) if indegree[i] == 0)
    done = 0
    while queue:
        node = queue.popleft()
        done += 1
        for nxt in graph[node]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
    return done == numCourses
```

How the solution works

1. The solution draws an edge from each prerequisite to the course that needs it. You can finish everything exactly when this directed graph has no cycle.
2. The in-degree of a course is the number of prerequisites it is still waiting for. A course with in-degree 0 can be taken right now.
3. The solution puts every course with in-degree 0 in a queue. It takes one, counts it, and removes its edges by lowering the in-degree of every course that depended on it.
4. Any course whose in-degree reaches 0 joins the queue. This method is Kahn's algorithm.
5. If the solution took all `numCourses` courses, there is no cycle. A course on a cycle never reaches in-degree 0, so `done` stays smaller than `numCourses`.

Time and space

Time: O(V + E). Building the graph takes O(V + E), each vertex enters the queue once, and each edge is followed once.

Auxiliary space: O(V + E). The adjacency lists hold E entries, and the in-degree list and the queue hold V.

Memory estimate

Each entry of `prerequisites` is a list of two integers, 72 bytes, and the course numbers above 256 are new integers. The adjacency lists hold the course integers that already exist in the input, the in-degree list holds the shared integers 0 and 1, and the queue holds one course at a time. Each adjacency list with one entry, built by `append`, is 88 bytes.

Part	How it is counted	Estimate
Input	1,999 pairs × 72 bytes, the outer list, and 3,485 course numbers above 256 × 28 bytes	252 KB
Auxiliary	the outer graph list, 56 + 8V; 1,999 one-entry lists × 88 bytes; the in-degree list, 56 + 8V; the deque, 760 bytes	204 KB
Total	input plus auxiliary	456 KB

Other approaches

Approach	Time	Auxiliary space
Depth-first search with three colors: unvisited, in progress and done	O(V + E)	O(V + E)

#210 Course Schedule II

Medium. https://leetcode.com/problems/course-schedule-ii/

The setup is the same as in Course Schedule: `prerequisites[i] = [a, b]` means that you must take course b before course a. Return one order in which you can take all the courses. If no such order exists, because the prerequisites contain a cycle, return an empty list.

For example, `numCourses = 4` and `prerequisites = [[1, 0], [2, 0], [3, 1], [3, 2]]` give `[0, 1, 2, 3]`; `[0, 2, 1, 3]` is also correct.

Start with `def findOrder(numCourses, prerequisites):`

Estimate the memory for V = 2,000 courses in one long chain of prerequisites, so E = 1,999.

Solution

```
from collections import deque

def findOrder(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    indegree = [0] * numCourses
    for course, pre in prerequisites:
        graph[pre].append(course)
        indegree[course] += 1
    queue = deque(i for i in range(numCourses) if indegree[i] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nxt in graph[node]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
    if len(order) == numCourses:
        return order
    return []
```

How the solution works

1. The solution builds the same graph and runs the same algorithm as Course Schedule, Kahn’s algorithm. The only change is that it records the courses in the order it takes them.
2. A course is appended to `order` when it leaves the queue. Every one of its prerequisites left the queue earlier, because the course entered the queue only when its in-degree reached 0.
3. The list `order` is therefore a valid order of the courses.
4. If `order` holds fewer than `numCourses` courses, some courses sit on a cycle, and the answer is an empty list.
5. More than one order can be correct, and any valid order is accepted.

Time and space

Time: O(V + E), as in Course Schedule.

Auxiliary space: O(V + E). The solution uses the same lists as Course Schedule plus the order list of V entries.

Memory estimate

The structures are the same as in Course Schedule, plus the order list, which holds V pointers to existing integers and is built by `append`.

Part	How it is counted	Estimate
Input	as in Course Schedule	252 KB
Auxiliary	as in Course Schedule (204 KB), plus the order list of 2,000 pointers (16 KB)	220 KB
Total	input plus auxiliary	472 KB

Other approaches

Approach	Time	Auxiliary space
Depth-first search, recording the vertices in reverse order of finishing	O(V + E)	O(V + E)

#743 Network Delay Time

Medium. https://leetcode.com/problems/network-delay-time/

A network has n nodes, labeled 1 to n. Each entry `times[i] = [u, v, w]` is a directed edge: a signal sent from u reaches v after w units of time. A signal is sent from node k. Return the time it takes for all n nodes to receive the signal, or −1 if some node can never receive it.

For example, `times = [[2, 1, 1], [2, 3, 1], [3, 4, 1]]`, `n = 4` and `k = 2` give 2.

Start with `def networkDelayTime(times, n, k):`

Estimate the memory for V = 100 nodes and E = 6,000 edges, which are LeetCode's limits, with random edges whose weights run from 1 to 100.

Solution

```
import heapq

def networkDelayTime(times, n, k):
    graph = {i: [] for i in range(1, n + 1)}
    for u, v, w in times:
        graph[u].append((v, w))
    dist = {}
    heap = [(0, k)]
    while heap:
        d, node = heapq.heappop(heap)
        if node in dist:
            continue
        dist[node] = d
        for nxt, w in graph[node]:
            if nxt not in dist:
                heapq.heappush(heap, (d + w, nxt))
    if len(dist) < n:
        return -1
    return max(dist.values())
```

How the solution works

1. The time the signal takes to reach a node is the shortest-path distance from k to that node, with the edge times as weights. The answer is the largest of those distances, the time at which the last node is reached.
2. Dijkstra's algorithm keeps a min-heap of (distance so far, node) pairs. The smallest entry is the next node whose distance is final.
3. The solution pops (d, node). If the node is already finished, which means it is already in `dist`, the entry is outdated and the solution skips it. Otherwise it sets `dist[node] = d`.
4. For each neighbor not yet finished, it pushes (d + w, neighbor). One node may have several entries on the heap, and only the cheapest one is used.
5. If fewer than n nodes were finished, some node cannot be reached, and the answer is −1. Otherwise the answer is the largest distance.

Time and space

Time: O(E log E), which is usually written O((V + E) log V). Each edge pushes at most one heap entry, each heap operation takes O(log E) time, and building the graph takes O(V + E).

Auxiliary space: O(V + E). The adjacency lists hold E edges, the heap holds at most E entries, and `dist` holds V entries.

Memory estimate

Each entry of `times` is a list of three integers, 56 + 3 × 8 = 80 bytes, and every number in it is below 257, so the numbers are shared. Each edge is stored once in the graph as a two-element tuple of 56 bytes plus an 8-byte slot, and the heap holds tuples of the same size; assume it peaks at about half the edges. Distances above 256 are new integers.

Part	How it is counted	Estimate
Input	6,000 lists × 80 bytes, plus the outer list, 56 + 8 × 6,000	516 KB
Auxiliary	the graph, 6,000 × 64 bytes; the heap, about 3,000 × 64 bytes; `dist`, 100 entries; and a few new integers	about 580 KB
Total	input plus auxiliary	1.1 MB

Other approaches

Approach	Time	Auxiliary space
Bellman-Ford: relax every edge V − 1 times	O(V × E)	O(V)
Breadth-first search, which works only when every weight is the same	O(V + E)	O(V)

#1631 Path With Minimum Effort

Medium. https://leetcode.com/problems/path-with-minimum-effort/

`heights` is a grid of integers. You start at the top-left cell and want to reach the bottom-right cell, moving up, down, left or right. The effort of a route is the largest absolute difference in height between two consecutive cells on the route. Return the smallest effort of any route.

For example, `heights = [[1, 2, 2], [3, 8, 2], [5, 3, 5]]` gives 2.

Start with `def minimumEffortPath(heights):`

Estimate the memory for a 100 × 100 grid, which is LeetCode's limit, so n = 10,000 cells, that is flat, with every height equal to 1 except at the goal, so that the search explores every cell at effort 0 before it reaches the goal.

Solution

```
import heapq

def minimumEffortPath(heights):
    rows = len(heights)
    cols = len(heights[0])
    best = [[float('inf')] * cols for _ in range(rows)]
    best[0][0] = 0
    heap = [(0, 0, 0)]
    while heap:
        effort, r, c = heapq.heappop(heap)
        if r == rows - 1 and c == cols - 1:
            return effort
        if effort > best[r][c]:
            continue
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                new = max(effort, abs(heights[nr][nc] - heights[r][c]))
                if new < best[nr][nc]:
                    best[nr][nc] = new
                    heapq.heappush(heap, (new, nr, nc))
```

How the solution works

1. Each cell is a vertex, and each move to a neighbor is an edge. The "distance" of a route is not a sum here but the largest step so far. Dijkstra's algorithm still works, because adding a step can never make the effort smaller.
2. `best[r][c]` is the smallest effort found so far to reach row r, column c. It starts at infinity everywhere except the start, where it is 0.
3. The solution pops the cell with the smallest effort from a min-heap. The first time it pops the goal, that effort is the answer.
4. For each neighbor, the effort of going through the current cell is `max(effort, abs(height difference))`. If that is better than the neighbor's best so far, the solution updates it and pushes the neighbor.
5. It skips outdated heap entries, whose effort is larger than the best known for their cell.

Time and space

Time: O(m × n × log(m × n)). Each cell can be pushed a few times, at most four, once for each improvement, and each push or pop costs O(log(m × n)).

Auxiliary space: O(m × n). The `best` table has m × n entries, and the heap holds a few entries per cell at most.

Memory estimate

The `best` table is 100 lists of 100 pointers to the shared integer 0 or the single float infinity. On this flat grid the heap holds only the edge of the search, about 200 entries, each a three-element tuple of 64 bytes plus an 8-byte slot. With heights designed to cause the most improvements, a cell can be pushed up to four times, and the heap could reach about 4n entries, about 3 MB.

Part	How it is counted	Estimate
Input	100 lists × (56 + 800), plus the outer list	84 KB
Auxiliary	the `best` table, 101 lists × 856 bytes, plus about 200 heap entries × 72 bytes	98 KB
Total	input plus auxiliary	182 KB

Other approaches

Approach	Time	Auxiliary space
Binary search on the answer, checking each guess with a breadth-first search	O(m × n × log H), where H is the largest height	O(m × n)
Sort the edges by height difference and join cells with union-find, from Class 22	O(m × n × log(m × n))	O(m × n)

#1334 Find the City With the Smallest Number of Neighbors at a Threshold Distance

Medium. https://leetcode.com/problems/find-the-city-with-the-smallest-number-of-neighbors-at-a-threshold-distance/

There are n cities, numbered 0 to n − 1. Each entry `edges[i] = [from, to, weight]` is a road in both directions between two cities, with the given length. The distance between two cities is the length of the shortest route between them. Return the city from which the fewest other cities lie within `distanceThreshold`. If several cities tie, return the one with the largest number.

For example, `n = 4`, `edges = [[0, 1, 3], [1, 2, 1], [1, 3, 4], [2, 3, 1]]` and `distanceThreshold = 4` give 3: cities 0 and 3 each reach two others, and 3 is the larger number.

Start with `def findTheCity(n, edges, distanceThreshold):`

Estimate the memory for n = 100 cities, which is LeetCode's limit, with a road between every pair, 4,950 roads, whose lengths run from 300 to 10,000.

Solution

```
def findTheCity(n, edges, distanceThreshold):
    INF = float('inf')
    dist = [[INF] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
    for a, b, w in edges:
        dist[a][b] = w
        dist[b][a] = w
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    best = -1
    fewest = n + 1
    for i in range(n):
        count = 0
        for j in range(n):
            if i != j and dist[i][j] <= distanceThreshold:
                count += 1
        if count <= fewest:
            fewest = count
            best = i
    return best
```

How the solution works

1. The problem needs the distance between every pair of cities, not just from one starting city. The Floyd-Warshall algorithm computes all of them at once.
2. The solution starts with a table in which `dist[i][j]` is the length of the direct road, 0 on the diagonal, and infinity where no road exists.
3. Floyd-Warshall then lets each city k in turn serve as a stopover. For every pair (i, j), if going through k is shorter than the best route known so far, the solution records the shorter distance. After city k has been tried, every distance in the table is the shortest route that uses only cities 0 to k as stopovers.
4. When all n cities have served as stopovers, the table holds every shortest distance. This is dynamic programming, with k as the stage.

5. Finally the solution counts, for each city, the other cities within the threshold. It uses <= when comparing counts, so a later city wins every tie, which returns the largest number as the problem asks.

Time and space

Time: $O(n^3)$. The three nested loops run n × n × n times, and filling the table and counting take $O(n^2)$ and O(E).

Auxiliary space: $O(n^2)$. The table holds n × n distances.

Memory estimate

The table is 100 lists of 100 pointers. Its entries start as the input's own road lengths or the single shared value of infinity, but every shorter route that the algorithm finds is a new integer, and in a dense graph most entries are replaced.

Part	How it is counted	Estimate
Input	4,950 roads × 80 bytes, the outer list, 56 + 8 × 4,950, and 4,950 lengths × 28 bytes	561 KB
Auxiliary	the table, 100 × (56 + 800) plus the outer list (84 KB), and up to 10,000 new distances × 28 bytes (273 KB)	up to 357 KB
Total	input plus auxiliary	up to 918 KB

Other approaches

Approach	Time	Auxiliary space
Dijkstra's algorithm from every city, which is faster when there are few roads	O(n × E log n)	O(n + E)
Bellman-Ford from every city	$O(n^2 \times E)$	O(n)

#787 Cheapest Flights Within K Stops

Medium. https://leetcode.com/problems/cheapest-flights-within-k-stops/

There are n cities. Each entry `flights[i] = [from, to, price]` is a one-way flight with the given price. Return the cheapest price from city `src` to city `dst` using at most k stops in between, or −1 if no such route exists.

For example, with `n = 4` and `flights = [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]]`, `src = 0`, `dst = 3` and `k = 1` give 700, by flying 0 to 1 to 3. The route 0 to 1 to 2 to 3 costs only 400, but it makes two stops.

Start with `def findCheapestPrice(n, flights, src, dst, k):`

Estimate the memory for n = 100 cities, which is LeetCode's limit, with 4,950 flights whose prices run from 300 to 10,000, and k = 99.

Solution

```
def findCheapestPrice(n, flights, src, dst, k):
    INF = float('inf')
    cost = [INF] * n
    cost[src] = 0
    for _ in range(k + 1):
        new_cost = cost[:]
        for a, b, price in flights:
            if cost[a] + price < new_cost[b]:
                new_cost[b] = cost[a] + price
        cost = new_cost
    if cost[dst] == INF:
        return -1
    return cost[dst]
```

How the solution works

1. A route with at most k stops uses at most k + 1 flights. The Bellman-Ford algorithm suits that limit, because after r rounds it has found the cheapest route of at most r edges.
2. `cost[v]` holds the cheapest known price to reach city v. It starts at 0 for `src` and infinity everywhere else.
3. Each round relaxes every flight: if reaching `a` and then flying to `b` is cheaper than the best known price for `b`, the solution records the cheaper price.
4. The solution reads from `cost`, the prices of the previous round, and writes into a copy, `new_cost`. Reading and writing the same list would let one round chain several flights together and break the limit on stops.
5. After k + 1 rounds, `cost[dst]` is the cheapest price with at most k stops, or infinity if `dst` cannot be reached in time.

Unlike Dijkstra's algorithm, Bellman-Ford also handles negative prices, and one more round that still improves a price reveals a negative cycle. Dijkstra's algorithm fails here for a different reason: the route that is cheapest so far may be the one that runs out of stops.

Time and space

Time: O(k × (n + E)). There are k + 1 rounds, and each copies n prices and relaxes all E flights.

Auxiliary space: O(n). The solution keeps two lists of n prices.

Memory estimate

Every flight is a list of three integers, and the prices above 256 are integers in the input. The two price lists are 856 bytes each, and the new totals the algorithm finds are new integers, at most one for each city in each list.

Part	How it is counted	Estimate
Input	4,950 flights × 80 bytes, the outer list, 56 + 8 × 4,950, and 4,950 prices × 28 bytes	561 KB
Auxiliary	two price lists of 100 pointers (1.7 KB) and up to 200 new totals × 28 bytes (5.5 KB)	about 7 KB
Total	input plus auxiliary	568 KB

Other approaches

Approach	Time	Auxiliary space
Dijkstra’s algorithm on (city, stops used) pairs	O(k × E × log(k × n))	O(k × n)
Breadth-first search by levels, one level per flight, keeping the cheapest price per city	O(k × E)	O(n + E)

Class 22: Union-find. Problem A, the easier problem.

#547 Number of Provinces

Medium. https://leetcode.com/problems/number-of-provinces/

There are n cities. `isConnected` is an n × n grid in which `isConnected[i][j]` is 1 if cities i and j are directly connected and 0 otherwise. A province is a group of cities connected to each other directly or through other cities. Return the number of provinces.

For example, `isConnected = [[1, 1, 0], [1, 1, 0], [0, 0, 1]]` gives 2, and `isConnected = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]` gives 3.

Start with `def findCircleNum(isConnected):`

Estimate the memory for n = 200 cities, which is LeetCode's limit, with no connections except each city to itself. This is the worst case for the scan.

Solution

```
def findCircleNum(isConnected):
    n = len(isConnected)
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    provinces = n
    for i in range(n):
        for j in range(i + 1, n):
            if isConnected[i][j] == 1:
                root_i = find(i)
                root_j = find(j)
                if root_i != root_j:
                    parent[root_i] = root_j
                    provinces -= 1
    return provinces
```

How the solution works

1. The solution starts with every city in its own group, with `parent[i] = i`, so each city is the root of its own group and there are n provinces.
2. `find(x)` follows the parent links up to the root. As it climbs, the line `parent[x] = parent[parent[x]]` points each city it passes at its grandparent, which shortens the chain and makes later finds faster. This step is called path halving.
3. For every connected pair (i, j), the solution finds both roots. If they differ, the groups are separate, so it joins them by pointing one root at the other and lowers the province count by one.
4. If the roots are the same, the two cities are already in the same province, and nothing changes.
5. The number of provinces left after all the pairs is the answer.

Time and space

Time: O(n²). Reading the upper half of the n × n grid takes about n²/2 steps, and each union-find operation takes nearly constant time on average.

Auxiliary space: O(n). The parent list holds n entries.

Memory estimate

The numbers 0 to 199 are shared small integers, so the parent list costs only its pointers. The grid is 200 lists of 200 pointers to the shared integers 0 and 1.

Part	How it is counted	Estimate
Input	200 lists × (56 + 1,600), plus the outer list	325 KB
Auxiliary	the parent list, 56 + 8n, plus a few integers	1.8 KB
Total	input plus auxiliary	327 KB

Other approaches

Approach	Time	Auxiliary space
Depth-first or breadth-first search over the grid	$O(n^2)$	O(n)

Class 22: Union-find. Problem B, the harder problem.

#684 Redundant Connection

Medium. https://leetcode.com/problems/redundant-connection/

A tree with n nodes, labeled 1 to n, had one extra edge added to it. The input is the resulting list of n edges, which contains exactly one cycle. Return an edge whose removal leaves a tree. If more than one edge works, return the one that appears last in the input.

For example, `edges = [[1, 2], [1, 3], [2, 3]]` gives `[2, 3]`, and `edges = [[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]]` gives `[1, 4]`.

Start with `def findRedundantConnection(edges):`

Estimate the memory for n = 1,000 edges, which is LeetCode's limit, forming a path 1–2–…–1,000, with the edge [1, 1,000] that closes the cycle last.

Solution

```
def findRedundantConnection(edges):
    parent = list(range(len(edges) + 1))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    for a, b in edges:
        root_a = find(a)
        root_b = find(b)
        if root_a == root_b:
            return [a, b]
        parent[root_a] = root_b
```

How the solution works

1. The solution adds the edges one at a time and keeps each connected group as a union-find tree.
2. An edge (a, b) whose ends are already in the same group closes a cycle, so that edge is the redundant one.
3. Otherwise the edge joins two groups, and the solution points one root at the other.
4. The first edge that closes a cycle is the last edge in the input that can be removed, because the input contains exactly one cycle and the solution stops at the edge that completes it.
5. Without union-find, a search from a to b before adding each edge would cost O(n) per edge and $O(n^2)$ in total.

Time and space

Time: nearly O(n). Each of the n edges needs two finds, and each find takes nearly constant time on average. The exact bound is $O(n \times \alpha(n))$, where α, the inverse Ackermann function, stays below 5 for any input you could store.

Auxiliary space: O(n). The parent list holds n + 1 entries.

Memory estimate

Each edge is a list of two integers, 72 bytes, and the node numbers above 256 are new integers. The parent list holds n + 1 pointers, and `list(range(...))` creates 744 new integers above 256.

Part	How it is counted	Estimate
Input	1,000 edges × 72 bytes, the outer list, and about 1,490 node numbers above 256 × 28 bytes	119 KB
Auxiliary	the parent list, 56 + 8 × 1,001, plus 744 new integers × 28 bytes	28 KB
Total	input plus auxiliary	147 KB

Other approaches

Approach	Time	Auxiliary space
Before adding each edge, search from a to see whether b is already reachable	$O(n^2)$	O(n)

#1584 Min Cost to Connect All Points

Medium. https://leetcode.com/problems/min-cost-to-connect-all-points/

You are given a list of points on a plane, `points[i] = [x, y]`. Connecting two points costs their Manhattan distance, $|x_1 - x_2| + |y_1 - y_2|$. Return the smallest total cost that connects all the points, so that there is exactly one path between any two of them.

For example, `points = [[0, 0], [2, 2], [3, 10], [5, 2], [7, 0]]` gives 20, and `points = [[3, 12], [-2, 5], [-4, 1]]` gives 18.

Start with `def minCostConnectPoints(points):`

Estimate the memory for n = 1,000 points, which is LeetCode's limit, with coordinates as large as 10^6, so count each coordinate as a new integer.

Solution

```
def minCostConnectPoints(points):
    n = len(points)
    INF = float('inf')
    best = [INF] * n
    best[0] = 0
    in_tree = [False] * n
    total = 0
    for _ in range(n):
        u = -1
        for v in range(n):
            if not in_tree[v] and (u == -1 or best[v] < best[u]):
                u = v
        in_tree[u] = True
        total += best[u]
        x, y = points[u]
        for v in range(n):
            if not in_tree[v]:
                d = abs(x - points[v][0]) + abs(y - points[v][1])
                if d < best[v]:
                    best[v] = d
    return total
```

How the solution works

1. Connecting every point at the lowest total cost, with exactly one path between any two, is a minimum spanning tree. Every pair of points can be joined, so the graph is complete, with $n(n - 1)/2$ possible edges.
2. Prim's algorithm grows the tree from one point. `best[v]` is the cheapest edge from point v to any point already in the tree.
3. At each step the solution adds the outside point with the smallest `best` value, and adds that cost to the total. Prim's algorithm is greedy: the cheapest edge leaving the tree is always safe to take.
4. It then updates `best` for every point still outside, because the new point may offer a cheaper connection.
5. In a complete graph this simple version, which scans for the minimum instead of using a heap, is the right choice: it takes $O(n^2)$ time without ever building the $n^2/2$ edges.

Time and space

Time: $O(n^2)$. There are n steps, and each scans all n points twice.

Auxiliary space: O(n). The solution keeps the `best` and `in_tree` lists.

Memory estimate

Each point is a list of two integers. The `best` list starts with the single shared infinity, and every distance it records is a new integer.

Part	How it is counted	Estimate
Input	the outer list, 56 + 8 × 1,000; 1,000 point lists × 72 bytes; 2,000 coordinates × 28 bytes	133 KB
Auxiliary	`best` and `in_tree`, 2 × (56 + 8 × 1,000), plus up to 1,000 new distances × 28 bytes	43 KB
Total	input plus auxiliary	176 KB

Other approaches

Approach	Time	Auxiliary space
Kruskal’s algorithm: build all n(n − 1)/2 edges, sort them, and join points with union-find	$O(n^2 \log n)$	$O(n^2)$
Prim’s algorithm with a heap of candidate edges	$O(n^2 \log n)$ on a complete graph	$O(n^2)$

#1489 Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree

Hard. https://leetcode.com/problems/find-critical-and-pseudo-critical-edges-in-minimum-spanning-tree/

You are given a connected, undirected graph with n vertices and a list of weighted edges, `edges[i] = [a, b, weight]`. A minimum spanning tree connects every vertex, without cycles, at the smallest total weight. An edge is critical if deleting it from the graph would raise that smallest weight. An edge is pseudo-critical if it appears in some minimum spanning trees but not in all of them. Return `[critical, pseudo]`, the indexes of the critical edges and of the pseudo-critical edges, each in any order.

For example, `n = 4` and `edges = [[0, 1, 1], [1, 2, 1], [2, 3, 1], [0, 3, 1]]` give `[[], [0, 1, 2, 3]]`: all four edges weigh the same, so any three of them form a minimum spanning tree, and no edge is in every one.

Start with `def findCriticalAndPseudoCriticalEdges(n, edges):`

Estimate the memory for n = 100 vertices and 200 edges, which are LeetCode's limits, with weights spread from 1 to 1,000.

Solution

```
   def findCriticalAndPseudoCriticalEdges(n, edges):
       order = sorted(range(len(edges)), key=lambda i: edges[i][2])
       def mst_weight(skip, force):
           parent = list(range(n))
           def find(x):
               while parent[x] != x:
                   parent[x] = parent[parent[x]]
                   x = parent[x]
               return x
           total = 0
           used = 0
           if force != -1:
               a, b, w = edges[force]
               parent[find(a)] = find(b)
               total += w
               used += 1
           for i in order:
               if i == skip:
                   continue
               a, b, w = edges[i]
               ra = find(a)
               rb = find(b)
               if ra != rb:
                   parent[ra] = rb
                   total += w
                   used += 1
           if used < n - 1:
               return float('inf')
           return total
       base = mst_weight(-1, -1)
       critical = []
       pseudo = []
       for i in range(len(edges)):
           if mst_weight(i, -1) > base:
               critical.append(i)
           elif mst_weight(-1, i) == base:
               pseudo.append(i)
       return [critical, pseudo]
```

How the solution works

1. Kruskal's algorithm builds a minimum spanning tree by taking the edges in order of weight and keeping each edge that joins two separate groups. Union-find, from Class 22, tells it whether two vertices are already in the same group.
2. The helper `mst_weight(skip, force)` runs Kruskal's algorithm with two options: it can leave out one edge, `skip`, or put one edge in first, `force`. If the edges it may use cannot connect every vertex, it returns infinity.
3. The solution first finds the smallest weight, `base`.
4. An edge is critical if leaving it out raises the weight above `base`, or disconnects the graph.
5. An edge that is not critical is pseudo-critical if forcing it in still gives weight `base`: some minimum spanning tree contains it, but not every one does. An edge that fails both tests appears in no minimum spanning tree.

The sort uses `sorted` with a `key`, which is built into Python and allowed at the board. Students who stall usually need the idea in step 2, that one Kruskal routine with two switches answers both questions.

Time and space

Time: O(E log E + E × (n + E × α(n))), where E is the number of edges. Sorting takes O(E log E). The routine runs up to 2E + 1 times, and each run resets n parents and makes up to E nearly constant-time finds.

Auxiliary space: O(n + E). The sorted order holds E indexes, each run holds n parents, and the two answer lists hold up to E indexes.

Memory estimate

Each edge is a list of three integers; the vertex numbers are below 257 and shared, and about three quarters of the weights are above 256. Only one run's parent list exists at a time.

Part	How it is counted	Estimate
Input	the outer list, 56 + 8 × 200; 200 edge lists × 80 bytes; about 150 weights × 28 bytes	21 KB
Auxiliary	the sorted order (1.6 KB), one parent list (856 bytes), the two answer lists, and a few frames	about 8 KB
Total	input plus auxiliary	29 KB

Other approaches

Approach	Time	Auxiliary space
List every spanning tree and keep the lightest ones, which works only for tiny graphs	exponential	exponential
Group the edges by weight and find bridges within each group with Tarjan's algorithm	O(E log E)	O(n + E)

#78 Subsets

Medium. https://leetcode.com/problems/subsets/

You are given a list of distinct integers, `nums`. Return every possible subset of the list, in any order, without repeating a subset.

For example, `nums = [1, 2, 3]` gives `[[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]`.

Start with `def subsets(nums):`

Estimate the memory for n = 10, which is LeetCode's limit, so there are 1,024 subsets with 5,120 entries in all. LeetCode's values run from −10 to 10.

Solution

```
def subsets(nums):
    result = []
    path = []
    def backtrack(start):
        result.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()
    backtrack(0)
    return result
```

How the solution works

1. A subset is a series of choices: for each number, in the order of the list, you take it or leave it. A list of n numbers therefore has 2^n subsets.
2. `path` holds the subset being built. Every path the search reaches is itself a valid subset, so each call copies it into the result. The slice `path[:]` makes the copy; without it, every entry of the result would be the same list.
3. The loop picks the next number to add. Starting at index `start` rather than 0 means the solution adds only numbers that come later in the list, so each subset is built in exactly one order and none repeats.
4. The solution chooses with `path.append`, explores with `backtrack(i + 1)`, and undoes the choice with `path.pop()`.
5. The tree of choices has 2^n nodes, and copying a path costs up to n.

Time and space

Time: O(n × 2^n). There are 2^n subsets, and copying each one into the result costs up to n.

Auxiliary space: O(n × 2^n). The returned subsets count, and together they hold n/2 × 2^n numbers. The search itself needs only O(n), for the recursion and the path.

Memory estimate

Every value from −10 to 10 is a shared small integer, so the numbers cost nothing extra and the subsets hold only pointers. Each subset is an exact-size copy, 56 bytes plus 8 per element.

Part	How it is counted	Estimate
Input	a list of 10 pointers, 56 + 80	136 bytes

Part	How it is counted	Estimate
Auxiliary	1,024 lists × 56 bytes plus 5,120 pointers × 8 bytes (96 KB), the result list of 1,024 pointers (8.6 KB), the path and about 10 frames	105 KB
Total	input plus auxiliary	105 KB

Other approaches

Approach	Time	Auxiliary space
Bitmasks: for each number from 0 to $2^n - 1$, take the elements whose bits are set	$O(n \times 2^n)$	$O(n \times 2^n)$ for the output
Start with `[[]]` and, for each number, add it to a copy of every subset so far	$O(n \times 2^n)$	$O(n \times 2^n)$ for the output

#51 N-Queens

Hard. https://leetcode.com/problems/n-queens/

Place n queens on an n × n chessboard so that no two queens attack each other, which means that no two share a row, a column or a diagonal. Return every distinct solution, each written as a list of n strings with `"Q"` for a queen and `"."` for an empty square.

For example, n = 4 gives `[[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]`.

Start with `def solveNQueens(n):`

Estimate the memory for n = 8, which has 92 solutions, each a list of 8 strings of 8 characters.

Solution

```
def solveNQueens(n):
    cols = set()
    diag1 = set()
    diag2 = set()
    placed = []
    result = []
    def place(r):
        if r == n:
            result.append(['.' * c + 'Q' + '.' * (n - c - 1) for c in placed])
            return
        for c in range(n):
            if c in cols or (r - c) in diag1 or (r + c) in diag2:
                continue
            cols.add(c)
            diag1.add(r - c)
            diag2.add(r + c)
            placed.append(c)
            place(r + 1)
            placed.pop()
            cols.remove(c)
            diag1.remove(r - c)
            diag2.remove(r + c)
    place(0)
    return result
```

How the solution works

1. Exactly one queen goes in each row, so the only choice at row r is the column. The solution never has to check rows.
2. Two queens clash when they share a column, the same c; a diagonal running down to the right, the same r − c; or a diagonal running down to the left, the same r + c. The solution keeps three sets of what is taken, so each check takes O(1) time.
3. `place(r)` tries every column c. If c, r − c or r + c is taken, it skips that column, which prunes the branch. Otherwise it adds all three to the sets, records c, and moves on to row r + 1.
4. To backtrack, it removes the column from `placed` and the three entries from the sets, so the next column starts clean.
5. When r equals n, every row has a queen, so the solution builds the board strings from `placed` and saves them.

Time and space

Time: O(n!) in the worst case. Row 0 has n choices, row 1 at most n − 1 safe columns, and so on. The pruning makes the real count far smaller.

Auxiliary space: O(S × n²), where S is the number of solutions. The returned boards count, and each holds n strings of n characters. The search itself needs only O(n), for the recursion and the three sets. At n = 8 there are 92 solutions, and at n = 9 there are 352.

Memory estimate

A string of 8 characters costs 41 + 8 = 49 bytes, and a solution list of 8 strings costs 56 + 64 bytes. The three sets hold at most 8 small integers each, about 730 bytes apiece. The call stack is 8 frames deep, at about 150 bytes each.

Part	How it is counted	Estimate
Input	one integer	28 bytes
Auxiliary	92 solutions × (120 + 8 × 49) bytes, the result list, 3 sets × 730 bytes and 8 frames × 150 bytes	50 KB
Total	input plus auxiliary	50 KB

Other approaches

Approach	Time	Auxiliary space
After each placement, scan the queen’s row, column and two diagonals for attacks	O(n × n!)	O(n²)

#208 Implement Trie (Prefix Tree)

Medium. https://leetcode.com/problems/implement-trie-prefix-tree/

Implement a trie. `insert(word)` stores a word; `search(word)` returns `True` if that exact word was inserted; and `startsWith(prefix)` returns `True` if any inserted word begins with `prefix`. Every string uses lowercase letters only.

For example, after `insert("apple")`, the call `search("apple")` returns `True`, `search("app")` returns `False` and `startsWith("app")` returns `True`; after `insert("app")`, `search("app")` returns `True`.

Start with `class Trie:` and write the methods `__init__(self)`, `insert(self, word)`, `search(self, word)` and `startsWith(self, prefix)`.

Estimate the memory for 10,000 random words of 8 lowercase letters each, 80,000 letters in all, created by the caller.

Solution

```
class Trie:
    def __init__(self):
        self.root = {}

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node['$'] = True

    def search(self, word):
        node = self.walk(word)
        return node is not None and '$' in node

    def startsWith(self, prefix):
        return self.walk(prefix) is not None

    def walk(self, text):
        node = self.root
        for ch in text:
            if ch not in node:
                return None
            node = node[ch]
        return node
```

How the solution works

1. A trie is a tree in which each edge is a letter, so the path from the root to a node spells a prefix. Words that share a prefix share the same path.
2. Each node is a dict that maps a letter to the child node. The special key `"$"` marks that a word ends at this node.
3. `insert` walks down the word, creating a child dict for any missing letter, and then sets `"$"` on the last node.
4. `walk(text)` follows a string from the root and returns the node where it ends, or `None` if the path breaks off. `search` needs that node to carry `"$"`, while `startsWith` needs only that the node exists.

5. Each operation costs O(L) for a word of length L, however many words are stored. A set of words would also find exact words in O(L) time, but it could not answer questions about prefixes without scanning every word.

Time and space

Time: O(L) per operation. Each operation touches one node for each letter.

Auxiliary space: O(total characters stored). The trie holds at most one node per stored character, and fewer when words share prefixes.

Memory estimate

Count the nodes level by level. Ten thousand random words fill the first two levels completely, 26 and 676 nodes, then about 7,630 nodes at depth 3 and nearly 10,000 at each depth from 4 to 8, about 58,200 nodes in all. Most nodes hold a single key, a letter or the end marker, and a dict with one string key costs 184 bytes; the nodes near the root hold many keys and cost more. Measured on Python 3.13, the trie averages about 187 bytes per node.

Part	How it is counted	Estimate
Input	10,000 words × (41 + 8) bytes, plus the list, 56 + 8 × 10,000	557 KB
Auxiliary	about 58,200 nodes × about 187 bytes	10.4 MB
Total	input plus auxiliary	11.0 MB

Other approaches

Approach	Time	Auxiliary space
A set of the words, scanning every word to answer `startsWith`	O(L) to insert or search; O(n × L) for `startsWith`	O(total characters)
A sorted list of the words, searched by binary search	O(n) to insert, because of the shift; O(L log n) to search or check a prefix	O(total characters)
Trie nodes as objects that hold a list of 26 child slots	O(L) per operation	more than the dict version, because every node reserves 26 slots

#212 Word Search II

Hard. https://leetcode.com/problems/word-search-ii/

You are given an m × n board of letters and a list of distinct words. Return every word from the list that can be spelled by a path of neighboring cells, moving horizontally or vertically. A path may use each cell at most once within the same word.

For example, `board = [["o","a","a","n"], ["e","t","a","e"], ["i","h","k","r"], ["i","f","l","v"]]` and `words = ["oath","pea","eat","rain"]` give `["eat", "oath"]`.

Start with `def findWords(board, words):`

Estimate the memory for 10,000 random words of 8 letters each, stored in a trie before the search starts, on a 12 × 12 board of random letters.

Solution

```
def findWords(board, words):
    root = {}
    for word in words:
        node = root
        for ch in word:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node['$'] = word
    rows = len(board)
    cols = len(board[0])
    found = []
    def dfs(r, c, parent):
        ch = board[r][c]
        node = parent[ch]
        if '$' in node:
            found.append(node.pop('$'))
        board[r][c] = '#'
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] in node:
                dfs(nr, nc, node)
        board[r][c] = ch
        if not node:
            parent.pop(ch)
    for r in range(rows):
        for c in range(cols):
            if board[r][c] in root:
                dfs(r, c, root)
    return found
```

How the solution works

1. Searching the board for one word at a time runs the backtracking search of Class 24 from every cell for each word, which repeats the same work for every word.
2. The solution puts all the words into one trie instead. A path on the board is worth following only while its letters spell a prefix that the trie holds, so a letter with no matching child in the trie cuts off the whole branch.
3. `dfs(r, c, parent)` steps onto a cell whose letter is a child of `parent`. If that trie node ends a word, the solution records the word and deletes the end marker, so the word is not found twice.

4. It marks the cell as used with `"#"`, searches the four neighbors, and then restores the letter. Restoring the letter is the backtracking step.
5. When a trie node has no children left, the solution removes it from its parent, so later searches never walk into a dead end.

The board solution returns `["oath", "eat"]`, because its scan reaches the `"o"` in the top-left corner first. Accept the words in any order. A version without the last pruning step, the removal of empty trie nodes, is also correct and is easier to write at the board.

Time and space

Time: $O(M \times 4 \times 3^{(L-1)})$, where M is the number of cells and L is the length of the longest word. From each starting cell the search can follow at most $4 \times 3^{(L-1)}$ paths of length up to L, because after the first step a path cannot turn back on itself. The trie cuts off most of those paths.

Auxiliary space: O(total letters in the words) for the trie, plus O(L) for the recursion.

Memory estimate

The trie is the peak. It is built from the same 10,000 random words as in Implement Trie, so it has about 58,200 nodes at about 187 bytes each. Each end marker stores a pointer to a word that the caller created.

Part	How it is counted	Estimate
Input	10,000 words × 49 bytes, the word list, and the board, 12 lists × (56 + 96) plus the outer list	559 KB
Auxiliary	about 58,200 trie nodes × about 187 bytes	10.4 MB
Total	input plus auxiliary	11.0 MB

Other approaches

Approach	Time	Auxiliary space
Run a separate board search for every word	$O(W \times M \times 4 \times 3^{(L-1)})$, where W is the number of words	O(L)

#55 Jump Game

Medium. https://leetcode.com/problems/jump-game/

`nums[i]` is the longest jump you can make from position i. Starting at index 0, return `True` if you can reach the last index, and `False` otherwise.

For example, `nums = [2, 3, 1, 1, 4]` gives `True`, and `nums = [3, 2, 1, 0, 4]` gives `False`.

Start with `def canJump(nums):`

Estimate the memory for n = 10,000 values, which is LeetCode's limit, all equal to 1, so that the reach grows one step at a time.

Solution

```
def canJump(nums):
    farthest = 0
    for i, jump in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + jump)
    return True
```

How the solution works

1. Trying every jump length from every position, by recursion or backtracking, takes exponential time, and even with memoization it takes $O(n^2)$.
2. The greedy idea is that it does not matter how you reach a position, only how far you can get from the positions you can reach. The solution keeps `farthest`, the farthest index reachable so far.
3. It scans from left to right. If position i lies beyond `farthest`, no earlier jump can reach it, and the answer is `False`.
4. Otherwise position i is reachable, and it extends the reach to `i + nums[i]`, so `farthest = max(farthest, i + jump)`.
5. If the scan finishes, every index was reachable, including the last one.

Time and space

Time: O(n). The solution makes one pass.

Auxiliary space: O(1). It keeps `farthest`, `i` and `jump`.

Memory estimate

Every value is the shared integer 1, so the input costs only its pointers.

Part	How it is counted	Estimate
Input	56 + 8n	78 KB
Auxiliary	`farthest`, `i`, `jump` and the `enumerate` object	about 200 bytes
Total	input plus auxiliary	78 KB

Other approaches

Approach	Time	Auxiliary space
Backtracking with memoization, which is dynamic programming	$O(n^2)$	$O(n)$

Class 26: Greedy algorithms and intervals. Problem B, the harder problem.

#135 Candy

Hard. https://leetcode.com/problems/candy/

n children stand in a line, and `ratings[i]` is the rating of child i. Give each child at least one candy, and give a child with a higher rating than a neighbor more candies than that neighbor. Return the smallest total number of candies.

For example, `ratings = [1, 0, 2]` gives 5, with candies 2, 1 and 2, and `ratings = [1, 2, 2]` gives 4, with candies 1, 2 and 1.

Start with `def candy(ratings):`

Estimate the memory for n = 50,000 children, which is LeetCode's limit, with ratings that strictly increase. This is the worst case for the size of the candy counts, which then run 1, 2, …, n.

Solution

```
def candy(ratings):
    n = len(ratings)
    candies = [1] * n
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            candies[i] = candies[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            candies[i] = max(candies[i], candies[i + 1] + 1)
    return sum(candies)
```

How the solution works

1. There are two rules, one for each neighbor. Satisfying both at once is awkward, so the solution satisfies them one at a time.
2. It starts every child at one candy.
3. In a pass from left to right, a child with a higher rating than the left neighbor gets one more candy than that neighbor. After this pass, every child is correct relative to the left neighbor.
4. In a pass from right to left, a child with a higher rating than the right neighbor needs at least one more candy than that neighbor. The solution keeps the larger of the old count and this new requirement, so the left rule still holds.
5. Each child ends with the smallest count that satisfies both neighbors, so the sum is the smallest total. Giving any child fewer candies would break a rule.

Time and space

Time: O(n). The solution makes two passes and one sum.

Auxiliary space: O(n). The candies list holds n entries. A solution that counts the rising and falling stretches of ratings needs only O(1) space.

Memory estimate

The ratings run from 0 to 49,999, so the 49,743 ratings above 256 are new integers. The candy counts run from 1 to 50,000, so the 49,744 counts above 256 are new integers as well.

Part	How it is counted	Estimate
Input	56 + 8n, plus 49,743 integers × 28 bytes	1.7 MB

Part	How it is counted	Estimate
Auxiliary	the candies list, 56 + 8n, plus 49,744 new counts × 28 bytes	1.7 MB
Total	input plus auxiliary	3.4 MB

Other approaches

Approach	Time	Auxiliary space
Two lists, one for each direction, taking the larger value at each child	O(n)	O(n)
Count the rising and falling stretches of ratings	O(n)	O(1)

#198 House Robber

Medium. https://leetcode.com/problems/house-robber/

`nums[i]` is the amount of money in house i along a street. You may not rob two neighboring houses. Return the largest amount you can rob.

For example, `nums = [1, 2, 3, 1]` gives 4, by robbing houses 0 and 2, and `nums = [2, 7, 9, 3, 1]` gives 12, by robbing houses 0, 2 and 4.

Start with `def rob(nums):`

Estimate the memory for n = 100 houses, which is LeetCode's limit, with values up to 400.

Solution

```
def rob(nums):
    prev = 0
    cur = 0
    for x in nums:
        prev, cur = cur, max(cur, prev + x)
    return cur
```

How the solution works

1. Define `best[i]` as the most money you can take from the first i houses. The ith house is `nums[i - 1]`, and for it there are two choices: skip it, which leaves `best[i - 1]`, or rob it, which gives `best[i - 2] + nums[i - 1]`, because the house before it is then off limits.
2. The recurrence is `best[i] = max(best[i - 1], best[i - 2] + nums[i - 1])`, with the base cases `best[0] = 0`, for no houses, and `best[1] = nums[0]`.
3. A table of n + 1 numbers works and takes O(n) space, but each entry looks back only two places.
4. So the solution keeps just two variables: `prev`, which is `best[i - 2]`, and `cur`, which is `best[i - 1]`.
5. At each house it computes the new value and shifts the pair with `prev, cur = cur, new`. The answer is `cur`.

Time and space

Time: O(n). The solution makes one pass and does a constant amount of work at each house.

Auxiliary space: O(1). It keeps two variables, while a full table would take O(n).

Memory estimate

The values above 256 and the running totals are new integers.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.6 KB
Auxiliary	`prev`, `cur`, `x` and a few temporary integers	about 150 bytes
Total	input plus auxiliary	3.7 KB

Other approaches

Approach	Time	Auxiliary space
Plain recursion without memory	$O(2^n)$	O(n)
A full table `best[0]` through `best[n]`	O(n)	O(n)

#322 Coin Change

Medium. https://leetcode.com/problems/coin-change/

`coins` is a list of coin values, with an unlimited supply of each, and `amount` is a target. Return the smallest number of coins that add up to exactly `amount`, or −1 if no combination does.

For example, `coins = [1, 2, 5]` and `amount = 11` give 3, from 5 + 5 + 1, and `coins = [2]` and `amount = 3` give −1.

Start with `def coinChange(coins, amount):`

Estimate the memory for `amount = 10,000`, which is LeetCode's limit, with `coins = [1, 2, 5]`.

Solution

```
def coinChange(coins, amount):
    INF = amount + 1
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a and dp[a - coin] + 1 < dp[a]:
                dp[a] = dp[a - coin] + 1
    if dp[amount] == INF:
        return -1
    return dp[amount]
```

How the solution works

1. A greedy rule fails here. With coins `[1, 3, 4]` and amount 6, taking the 4 first gives 4 + 1 + 1, three coins, but 3 + 3 uses only two.
2. Define `dp[a]` as the smallest number of coins that make the amount a. The base case is `dp[0] = 0`.
3. The last coin used is some coin c, so `dp[a]` is 1 plus the smallest `dp[a - c]` over every coin c no larger than a.
4. The solution fills a from 1 up to `amount`, so `dp[a - c]` is always known when it is needed. `INF = amount + 1` stands for "impossible", because it is larger than any real answer.
5. If `dp[amount]` is still `INF` at the end, no combination works, and the answer is −1.

Time and space

Time: O(amount × number of coins). There are amount + 1 states, and each tries every coin.

Auxiliary space: O(amount). The `dp` list holds amount + 1 entries.

Memory estimate

The coins 1, 2 and 5 are shared small integers. The `dp` list is built by concatenation, so it has exactly amount + 1 slots. The counts reach 2,000, and every count above 256, about 87 percent of the entries here, is a new integer.

Part	How it is counted	Estimate
Input	a list of 3 pointers, 56 + 24	80 bytes
Auxiliary	the `dp` list, 56 + 8 × 10,001, plus about 8,700 new counts × 28 bytes	316 KB
Total	input plus auxiliary	316 KB

Other approaches

Approach	Time	Auxiliary space
Recursion without memory	exponential in `amount`	O(amount)
Breadth-first search over amounts	O(amount × number of coins)	O(amount)
A greedy rule, which gives wrong answers in general	not applicable	not applicable

#300 Longest Increasing Subsequence

Medium. https://leetcode.com/problems/longest-increasing-subsequence/

You are given a list of integers, `nums`. Return the length of its longest strictly increasing subsequence. A subsequence keeps the order of the numbers, but its numbers need not be next to each other.

For example, `nums = [10, 9, 2, 5, 3, 7, 101, 18]` gives 4, for example from 2, 3, 7, 101.

Start with `def lengthOfLIS(nums):`

Estimate the memory for n = 2,500 values, which is LeetCode's limit, strictly increasing. This is the worst case for the values in `dp`, which then run 1, 2, …, n.

Solution

```
def lengthOfLIS(nums):
    n = len(nums)
    dp = [1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i] and dp[j] + 1 > dp[i]:
                dp[i] = dp[j] + 1
    return max(dp)
```

How the solution works

1. Define `dp[i]` as the length of the longest increasing subsequence that ends exactly at position i. A state such as "the best in the first i numbers" would not say whether the next number can extend it, which is why the state ends at i.
2. Every position on its own is a subsequence of length 1, so `dp` starts at 1 everywhere.
3. The recurrence is `dp[i] = 1 + max(dp[j])` over the earlier positions j with `nums[j] < nums[i]`. The solution tries every earlier j.
4. The answer is the best length ending anywhere, `max(dp)`.
5. There are n states, and each looks at up to n earlier values, which gives $O(n^2)$ time and O(n) space. A faster solution, O(n log n), keeps a list of the smallest possible tail of each length and searches it by binary search.

Time and space

Time: $O(n^2)$. For each i the inner loop runs i times, which adds up to $n(n - 1)/2$ steps.

Auxiliary space: O(n). The `dp` list holds n entries.

Memory estimate

The values above 256 in the input, and the lengths above 256 in `dp`, are new integers.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	88 KB
Auxiliary	the `dp` list, 56 + 8n, plus 28 bytes for each of the n − 256 lengths above 256	81 KB
Total	input plus auxiliary	169 KB

Other approaches

Approach	Time	Auxiliary space
A list of tails searched by binary search, written by hand or with `bisect_left`, which is not allowed at the board	O(n log n)	O(n)
Plain recursion	$O(2^n)$	O(n)

Class 28: Dynamic programming: sequences. Problem B, the harder problem.

#1143 Longest Common Subsequence

Medium. https://leetcode.com/problems/longest-common-subsequence/

You are given two strings, `text1` and `text2`. Return the length of their longest common subsequence, the longest sequence of characters that appears in both strings in the same order, though not necessarily next to each other. Return 0 if they have none.

For example, `text1 = "abcde"` and `text2 = "ace"` give 3, from `"ace"`, and `text1 = "abc"` and `text2 = "def"` give 0.

Start with `def longestCommonSubsequence(text1, text2):`

Estimate the memory for two random strings of 1,000 lowercase letters each, which is LeetCode's limit.

Solution

```
def longestCommonSubsequence(text1, text2):
    m = len(text1)
    n = len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

How the solution works

1. Define `dp[i][j]` as the length of the longest common subsequence of the first i characters of `text1` and the first j characters of `text2`. Row 0 and column 0 are 0, because an empty string has nothing in common with anything.
2. If the last characters, `text1[i - 1]` and `text2[j - 1]`, are equal, they extend the best common subsequence of the two shorter prefixes, so the cell is `dp[i - 1][j - 1] + 1`.
3. If they differ, at least one of them is not used, so the cell takes the better of dropping the last character of `text1`, `dp[i - 1][j]`, or of `text2`, `dp[i][j - 1]`.
4. The solution fills the table row by row. Each cell needs only the cell above it, the cell to its left and the cell diagonally above and to the left, all of which are already filled.
5. The answer is the bottom-right cell. Keeping only the previous row would cut the space to O(n).

This is the course's first two-dimensional table, one class before Class 29 makes such tables the main subject.

Time and space

Time: O(m × n). The table has m × n cells, and each takes constant time.

Auxiliary space: O(m × n). The table has (m + 1) × (n + 1) cells.

Memory estimate

The table is 1,001 lists of 1,001 pointers. For two random strings of 1,000 letters the answer is about 320, so only a few cells hold counts above 256, which are new integers; they add about 0.1 MB.

Part	How it is counted	Estimate
Input	2 strings × (41 + 1,000) bytes	2.0 KB
Auxiliary	the table, 1,001 × (56 + 8 × 1,001) bytes (7.7 MB), plus new counts above 256 (about 0.1 MB)	7.8 MB
Total	input plus auxiliary	7.8 MB

Other approaches

Approach	Time	Auxiliary space
Plain recursion without memory	$O(2^{(m + n)})$	O(m + n)
Keep only two rows of the table	O(m × n)	O(min(m, n))

#72 Edit Distance

Medium. https://leetcode.com/problems/edit-distance/

You are given two words, `word1` and `word2`. Return the smallest number of single-character operations, each an insertion, a deletion or a replacement, that turn `word1` into `word2`.

For example, `word1 = "horse"` and `word2 = "ros"` give 3: horse becomes rorse, then rose, then ros.

Start with `def minDistance(word1, word2):`

Estimate the memory for two random words of 500 lowercase letters each, which is LeetCode's limit.

Solution

```
def minDistance(word1, word2):
    m = len(word1)
    n = len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
    return dp[m][n]
```

How the solution works

1. Define `dp[i][j]` as the smallest number of operations that turn the first i characters of `word1` into the first j characters of `word2`.
2. The base cases come from the empty string. Turning i characters into nothing takes i deletions, so `dp[i][0] = i`, and turning nothing into j characters takes j insertions, so `dp[0][j] = j`.
3. If the last characters match, they need no operation, so `dp[i][j] = dp[i - 1][j - 1]`.
4. If they differ, the solution tries the three operations and takes the cheapest one plus one: deleting from `word1`, `dp[i - 1][j]`; inserting into `word1`, `dp[i][j - 1]`; or replacing, `dp[i - 1][j - 1]`.
5. The answer is `dp[m][n]`. Filling three cells by hand for `"ab"` and `"b"` is a quick check of the recurrence.

Time and space

Time: O(m × n). The table has m × n cells, and each takes constant time.

Auxiliary space: O(m × n). The table has (m + 1) × (n + 1) cells. Two rows are enough if you do not need to recover the edits themselves.

Memory estimate

The table is 501 lists of 501 pointers. For two random 500-letter words the distance is about 440, and about two thirds of the 250,000 cells hold a value above 256, each a new integer.

Part	How it is counted	Estimate
Input	2 strings × (41 + 500) bytes	1.1 KB
Auxiliary	the table, 501 × (56 + 8 × 501) bytes (1.9 MB), plus a new integer of 28 bytes in about 0.65 × 250,000 cells (4.3 MB)	6.3 MB
Total	input plus auxiliary	6.3 MB

Other approaches

Approach	Time	Auxiliary space
Plain recursion without memory	$O(3^{(m + n)})$	O(m + n)
Keep only two rows of the table	O(m × n)	O(min(m, n))

#10 Regular Expression Matching

Hard. https://leetcode.com/problems/regular-expression-matching/

Implement pattern matching in which `"."` matches any single character and `"*"` matches zero or more copies of the character just before it. Return `True` if the pattern `p` matches the whole string `s`, and `False` otherwise.

For example, `s = "aa"` and `p = "a"` give `False`, `s = "aa"` and `p = "a*"` give `True`, and `s = "ab"` and `p = ".*"` give `True`.

Start with `def isMatch(s, p):`

Estimate the memory for a string and a pattern of 20 characters each, which is LeetCode's limit.

Solution

```
def isMatch(s, p):
    m = len(s)
    n = len(p)
    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True
    for j in range(2, n + 1):
        if p[j - 1] == '*':
            dp[0][j] = dp[0][j - 2]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if p[j - 1] == '*':
                dp[i][j] = dp[i][j - 2]
                if p[j - 2] == s[i - 1] or p[j - 2] == '.':
                    dp[i][j] = dp[i][j] or dp[i - 1][j]
            elif p[j - 1] == s[i - 1] or p[j - 1] == '.':
                dp[i][j] = dp[i - 1][j - 1]
    return dp[m][n]
```

How the solution works

1. Define `dp[i][j]` as `True` if the first i characters of `s` match the first j characters of `p`. `dp[0][0]` is `True`, because the empty pattern matches the empty string.
2. In row 0, an empty string matches a pattern only if every starred item can be used zero times, so `dp[0][j] = dp[0][j - 2]` whenever `p[j - 1]` is `"*"`.
3. For an ordinary character or a `"."`, `dp[i][j]` is `True` when the last characters agree, meaning `p[j - 1]` equals `s[i - 1]` or is `"."`, and `dp[i - 1][j - 1]` is `True`.
4. A star at `p[j - 1]` acts on the character before it, `p[j - 2]`. Using zero copies gives `dp[i][j - 2]`. Using one more copy is possible only if `p[j - 2]` matches `s[i - 1]` or is `"."`, and it gives `dp[i - 1][j]`, because the star can still absorb further copies.
5. The answer is `dp[m][n]`. Tracing `"aa"` against `"a*"` in a 3 × 3 table checks all three rules.

Time and space

Time: O(m × n). The table has (m + 1) × (n + 1) cells, and each takes constant time.

Auxiliary space: O(m × n). The table holds that many booleans, and `True` and `False` are shared objects.

Memory estimate

Part	How it is counted	Estimate
Input	2 strings × (41 + 20) bytes	122 bytes

Part	How it is counted	Estimate
Auxiliary	the table, 21 lists × (56 + 8 × 21) bytes, plus the outer list and a few integers	4.9 KB
Total	input plus auxiliary	5.0 KB

Other approaches

Approach	Time	Auxiliary space
Recursion without memory	exponential	O(m + n)
Recursion from the top with a memo	O(m × n)	O(m × n)

#127 Word Ladder

Hard. https://leetcode.com/problems/word-ladder/

`beginWord` and `endWord` are words of the same length, and `wordList` is a list of allowed words. A move changes exactly one letter, and every word you reach must be in `wordList`. Return the number of words in the shortest sequence of moves from `beginWord` to `endWord`, counting both ends, or 0 if no such sequence exists.

For example, `beginWord = "hit"`, `endWord = "cog"` and `wordList = ["hot","dot","dog","lot","log","cog"]` give 5: hit, hot, dot, dog, cog.

Start with `def ladderLength(beginWord, endWord, wordList):`

Estimate the memory for N = 3,125 words, every 5-letter word made from the letters a to e, with L = 5, `beginWord = "aaaaa"` and `endWord = "eeeee"`.

Solution

```
from collections import deque

def ladderLength(beginWord, endWord, wordList):
    words = set(wordList)
    if endWord not in words:
        return 0
    queue = deque([(beginWord, 1)])
    seen = {beginWord}
    while queue:
        word, steps = queue.popleft()
        if word == endWord:
            return steps
        for i in range(len(word)):
            for ch in 'abcdefghijklmnopqrstuvwxyz':
                nxt = word[:i] + ch + word[i + 1:]
                if nxt in words and nxt not in seen:
                    seen.add(nxt)
                    queue.append((nxt, steps + 1))
    return 0
```

How the solution works

1. The problem is a graph. Each word is a vertex, and two words are joined by an edge when they differ in one letter. The answer is the shortest path from `beginWord` to `endWord` measured in edges, plus one to count words, and a breadth-first search finds it.
2. The solution never builds the graph. It generates the neighbors of a word when it needs them, by changing each position to each of the 26 letters and keeping the results that are in the word set, which a set answers in O(1) time.
3. It runs a breadth-first search from `beginWord` with a queue of (word, number of words so far) pairs. The first time it takes `endWord` out of the queue, `steps` is the answer.
4. It marks a word as seen when it adds the word to the queue, so no word is queued twice.
5. If the queue empties, `endWord` cannot be reached, and the answer is 0.

Time and space

Time: O(N × L² × 26), where N is the number of words and L is the word length. Each word leaves the queue once, the solution tries L positions × 26 letters for it, and building each candidate string costs O(L).

Auxiliary space: O(N × L). The word set, the seen set and the queue hold up to N words of L letters each.

Memory estimate

The word set and the seen set each cost about 42 bytes per word. Each seen word is a new string made by the search, 41 + 5 bytes, and the queue holds two-element tuples of 56 bytes plus an 8-byte slot, with step counts that are shared small integers. The peak comes at about the point where the search has used up half the graph.

Part	How it is counted	Estimate
Input	3,125 words × (41 + 5) bytes, plus the list	165 KB
Auxiliary	the word set, 42N; the seen set, 42N; new strings, 46N; queue tuples, about 36N on average	507 KB
Total	input plus auxiliary	671 KB

Other approaches

Approach	Time	Auxiliary space
Breadth-first search from both ends at once	about $O(N \times L^2 \times 26)$, and usually much less	$O(N \times L)$
Wildcard buckets that map a pattern such as `"h*t"` to the words that fit it	$O(N \times L^2)$	$O(N \times L^2)$

#295 Find Median from Data Stream

Hard. https://leetcode.com/problems/find-median-from-data-stream/

Design a class `MedianFinder`. Its method `addNum(num)` adds an integer from a stream of data, and `findMedian()` returns the median of all the numbers so far: the middle value if the count is odd, or the mean of the two middle values if the count is even.

For example, after `addNum(1)` and `addNum(2)`, `findMedian()` returns 1.5; after `addNum(3)`, it returns 2.0.

Start with `class MedianFinder:` and write the methods `__init__(self)`, `addNum(self, num)` and `findMedian(self)`.

Estimate the memory for n = 50,000 numbers added, which is LeetCode's limit.

Solution

```
import heapq

class MedianFinder:
    def __init__(self):
        self.small = []
        self.large = []

    def addNum(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self):
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2
```

How the solution works

1. Keeping the list sorted costs O(n) for each insertion, because of the shifting, and sorting at every query costs O(n log n). Yet the solution only ever needs the middle.
2. It splits the numbers into two halves: `small` holds the smaller half and `large` holds the larger half. The median then sits at the boundary between them.
3. `small` must give up its largest value quickly, so it needs a max-heap. Python has only a min-heap, so `small` stores negated values. `large` is an ordinary min-heap, which gives up its smallest value.
4. Two facts hold after every call. Every value in `small` is at most every value in `large`, and `small` holds as many values as `large` or one more.
5. `addNum` pushes into `small`, moves the largest value of `small` over to `large`, which keeps the order, and moves the smallest value of `large` back if `large` has grown bigger. `findMedian` returns the top of `small` if the count is odd, and otherwise the mean of the two tops.

Time and space

Time: O(log n) for `addNum` and O(1) for `findMedian`. `addNum` always does three heap operations and, when it rebalances, two more, each on a heap of at most n/2 + 1 values. `findMedian` reads the two tops.

Auxiliary space: O(n). Every number is stored once, in one of the two heaps.

Memory estimate

Both heaps hold new integers, made by negating values on the way in and out, at 28 bytes each. The two lists have 8-byte slots with a few percent of spare room. The caller's own integers are not kept.

Part	How it is counted	Estimate
Input	a stream of n integers, 28n	1.3 MB
Auxiliary	n new integers, 28n, plus two lists of about 9n together, plus 2 × 56	1.8 MB
Total	input plus auxiliary	3.1 MB

Other approaches

Approach	Time	Auxiliary space
A sorted list, inserting each number at its place found by binary search	O(n) per `addNum`, because of the shift; O(1) per `findMedian`	O(n)
An unsorted list, sorted at every `findMedian`	O(1) per `addNum`; O(n log n) per `findMedian`	O(n)
Two heaps (the solution above)	O(log n) per `addNum`; O(1) per `findMedian`	O(n)